

DevOps Shack

50 DevOps Projects

Best Use Cases, Tools, and Implementations

Covering All Aspects of Modern DevOps

Author

Aditya Jaiswal
[@devopsshack](#)
[devopsshack.com](#)

Document Details

Version: 1.0
Projects Covered: 50
Categories: 12+

From beginner-friendly setups to enterprise-grade architectures — your complete DevOps project guide

Table of Contents

How to Use This Guide	4
Project Categories.....	4
Project 1: Automated CI/CD Pipeline with Jenkins	5
Project 2: Infrastructure as Code with Terraform on AWS.....	7
Project 3: Kubernetes Cluster Setup and Application Deployment	9
Project 4: Docker Containerisation of a Multi-Tier Application	11
Project 5: GitOps Workflow with ArgoCD.....	13
Project 6: Monitoring and Alerting with Prometheus and Grafana.....	15
Project 7: Centralised Log Management with ELK Stack	17
Project 8: Secrets Management with HashiCorp Vault.....	19
Project 9: Blue-Green Deployments on AWS	21
Project 10: Canary Deployments with Flagger and Istio.....	23
Project 11: DevSecOps Pipeline with SAST, DAST, and SCA.....	25
Project 12: Multi-Cloud Infrastructure Management	27
Project 13: Automated Database Schema Migration Pipeline	29
Project 14: Service Mesh Implementation with Istio	31
Project 15: Cost Optimisation Dashboard for Cloud Spend	33
Project 16: Automated Incident Response with PagerDuty and Runbooks.....	35
Project 17: Container Image Hardening and Vulnerability Management.....	37
Project 18: Platform Engineering: Internal Developer Platform (IDP).....	39
Project 19: Chaos Engineering with LitmusChaos.....	41
Project 20: Serverless CI/CD Pipeline on AWS with CodePipeline	43
Project 21: Container Registry and Image Lifecycle Management.....	45
Project 22: Distributed Tracing with Jaeger and OpenTelemetry	47
Project 23: Automated Compliance as Code with Open Policy Agent	49
Project 24: Feature Flag Management System	51
Project 25: Automated Performance Testing Pipeline	53
Project 26: Event-Driven Architecture with Kafka and Kubernetes	55
Project 27: Disaster Recovery and Business Continuity Planning	57
Project 28: Kubernetes Multi-Tenancy with Namespace Isolation	59
Project 29: Immutable Infrastructure with Packer and AWS AMIs	61
Project 30: Kubernetes Cost Optimisation with Vertical and Horizontal Autoscaling	63
Project 31: API Gateway and Rate Limiting with Kong.....	65
Project 32: Continuous Security Scanning with Falco Runtime Security.....	67
Project 33: GitLab CI/CD Self-Hosted Pipeline.....	69
Project 34: Zero-Trust Network Security Implementation	71
Project 35: Cloud-Native Backup and Data Protection	73
Project 36: Developer Environment Automation with Dev Containers	75
Project 37: SRE Practices: SLIs, SLOs, and Error Budgets	77

Project 38: Automated Kubernetes Cluster Upgrades	79
Project 39: Multi-Region Active-Active Application Deployment.....	81
Project 40: Automated Terraform Drift Detection and Remediation	83
Project 41: DataOps Pipeline for Analytics Infrastructure	85
Project 42: Microservices Communication with gRPC and Protobuf	87
Project 43: Cloud Security Posture Management (CSPM) Implementation	89
Project 44: AI/ML Model Deployment with MLflow and Kubernetes	91
Project 45: Network Policy Automation with Cilium eBPF	93
Project 46: Automated Certificate Management with cert-manager	95
Project 47: Automated Compliance Reporting for SOC 2 / ISO 27001	97
Project 48: Progressive Web App Deployment Pipeline	99
Project 49: Automated Kubernetes Security Scanning with Kube-bench and Kube-hunter	101
Project 50: Full-Stack Observability with OpenTelemetry and Grafana Stack	103

About This Document

This document is a comprehensive reference guide covering 50 real-world DevOps projects. Each project provides a description, detailed use case, complete toolset, step-by-step implementation guide, best practices, and a section addressing common challenges and their solutions. The guide is structured to be accessible to beginners while providing the depth experienced engineers need.

Projects span the full spectrum of DevOps: from foundational CI/CD pipelines and infrastructure automation through to advanced topics such as chaos engineering, MLOps, zero-trust security, and multi-region active-active architectures.

How to Use This Guide

Read sequentially to build understanding progressively, or jump directly to the project relevant to your current challenge. Each project is self-contained. Use the Table of Contents to navigate by project number or category.

Project Categories

Category	Projects
Continuous Integration & Delivery	Projects 1, 20, 33, 48
Infrastructure Automation	Projects 2, 12, 29, 40
Container Orchestration	Projects 3, 28, 38
Containerisation	Projects 4, 17, 21
GitOps	Project 5
Observability	Projects 6, 7, 22, 50
Security & DevSecOps	Projects 8, 11, 23, 32, 34, 43, 46, 47, 49
Deployment Strategies	Projects 9, 10, 24
FinOps	Projects 15, 30
Reliability Engineering	Projects 19, 27, 35, 37, 39
Platform Engineering	Project 18
Incident Management	Project 16
Database DevOps	Project 13
Networking & Service Mesh	Projects 14, 31, 42, 45
DataOps & MLOps	Projects 41, 44
Developer Experience	Projects 36
Messaging & Event Streaming	Project 26

Project 1: Automated CI/CD Pipeline with Jenkins

Project 1 of 50 • Continuous Integration & Delivery

Description

Build a fully automated software delivery pipeline using Jenkins that compiles code, runs tests, performs static analysis, and deploys to multiple environments. The pipeline covers every stage from a developer pushing code to production deployment, reducing manual effort and increasing release frequency.

Use Case

Software teams that release frequently and need reliable, repeatable builds. Ideal for microservices architectures where dozens of services are updated daily and manual deployment would be error-prone and time-consuming.

Tools and Technologies

Jenkins, Git, Maven/Gradle, JUnit, SonarQube, Nexus Repository, Docker, Ansible, Slack (notifications)

Implementation Steps

1. Install Jenkins on a dedicated server or cloud VM; configure JDK, Maven, and Git plugins.
2. Create a Jenkinsfile in the root of your repository defining pipeline stages: Checkout, Build, Unit Test, Code Analysis, Artifact Publish, Deploy to Staging, Integration Test, Deploy to Production.
3. Configure SonarQube integration for static code quality gates; fail the pipeline if quality thresholds are not met.
4. Set up Nexus as the artifact repository; publish versioned JARs/WARs after a successful build.
5. Write Ansible playbooks to deploy artifacts to staging and production servers; parameterise environment-specific variables.
6. Enable Slack notifications for pipeline success, failure, and manual approval steps.
7. Implement branch protection rules in Git: only allow merges to main after the pipeline passes.
8. Schedule nightly regression pipelines against the staging environment.

Best Practices

- Keep the Jenkinsfile in version control alongside application code so pipeline changes follow the same review process.
- Use parameterised builds to promote the same artifact across environments, never rebuild from source for each stage.
- Set strict quality gates in SonarQube; treat code coverage drops as pipeline failures.
- Cache Maven/Gradle dependencies between runs to cut build times by 40–60%.
- Use pipeline-as-code to make the entire workflow auditable and reproducible.

Challenges and Solutions

Challenge	Solution
Long build times slow developer feedback loops.	Parallelise test stages and cache dependencies. Use agent labels to direct heavy stages to more powerful nodes.

Challenge

Flaky tests cause intermittent pipeline failures.

Solution

Tag flaky tests, quarantine them, and run them in a separate non-blocking stage while the fix is tracked in the backlog.

Challenge

Secrets (passwords, API keys) leak into build logs.

Solution

Store credentials in Jenkins Credential Store or HashiCorp Vault; never pass secrets as plain environment variables.

Project 2: Infrastructure as Code with Terraform on AWS

Project 2 of 50 • Infrastructure Automation

Description

Provision and manage an entire AWS infrastructure stack — VPCs, subnets, EC2 instances, RDS databases, S3 buckets, and IAM roles — using Terraform. The entire setup is versioned in Git, enabling infrastructure changes to follow the same review-and-approval workflow as application code.

Use Case

Teams building cloud-native applications that need consistent, repeatable environments across development, staging, and production. Eliminates configuration drift and enables disaster recovery by recreating infrastructure from code within minutes.

Tools and Technologies

Terraform, AWS (VPC, EC2, RDS, S3, IAM, Route 53), GitHub Actions, AWS S3 (remote state), DynamoDB (state lock), pre-commit hooks, tfint, checkov

Implementation Steps

1. Organise Terraform code into modules: networking, compute, database, security.
2. Create an S3 bucket and DynamoDB table for remote state storage and locking.
3. Write root module variables for environment (dev/staging/prod) and region.
4. Define VPC with public and private subnets across two availability zones.
5. Provision EC2 instances with launch templates and Auto Scaling Groups.
6. Set up RDS in a private subnet with Multi-AZ for production environments.
7. Integrate tfint and checkov into a pre-commit hook to catch misconfigurations before push.
8. Create a GitHub Actions workflow: terraform fmt, validate, plan on PR; apply on merge to main.

Best Practices

- Use workspaces or separate state files per environment to prevent accidental cross-environment changes.
- Tag every resource with environment, owner, cost-centre, and managed-by=terraform.
- Pin provider and module versions to avoid unexpected breaking changes.
- Review the terraform plan output in pull requests using automated PR comments.
- Enable drift detection by running terraform plan on a schedule and alerting on differences.

Challenges and Solutions

Challenge

State file conflicts in team environments.

Solution

Enforce state locking with DynamoDB. Educate the team never to run apply locally against shared state without coordination.

Challenge

Destroying production resources by accident.

Solution

Use lifecycle { prevent_destroy = true } on critical resources. Require manual approval in the pipeline before production apply.

Challenge

Module sprawl making code hard to navigate.

Solution

Establish a module registry with versioned releases. Document each module's inputs, outputs, and usage examples.

Project 3: Kubernetes Cluster Setup and Application Deployment

Project 3 of 50 • Container Orchestration

Description

Set up a production-grade Kubernetes cluster on AWS EKS, configure networking, storage, and RBAC, then deploy a multi-tier web application using Helm charts. The setup includes horizontal pod autoscaling, rolling updates, and health checks to ensure high availability.

Use Case

Companies migrating from monolithic or VM-based deployments to containers. Essential for applications with variable traffic that benefit from automatic scaling and for teams running multiple services that share compute resources.

Tools and Technologies

AWS EKS, kubectl, Helm, eksctl, AWS Load Balancer Controller, EBS CSI Driver, Prometheus, Grafana, cert-manager, External-DNS

Implementation Steps

1. Create an EKS cluster with managed node groups using eksctl; specify instance types and desired/min/max node counts.
2. Install the AWS Load Balancer Controller for Ingress support using Helm and IRSA.
3. Configure the EBS CSI Driver for persistent volume support; create StorageClasses.
4. Set up namespaces for each environment (dev, staging, prod) with ResourceQuotas.
5. Write Helm charts for the frontend, backend, and database services with configurable values.
6. Configure Horizontal Pod Autoscaler based on CPU and custom metrics.
7. Install cert-manager and configure Let's Encrypt ClusterIssuer for automatic TLS.
8. Deploy Prometheus and Grafana for cluster and application metrics.

Best Practices

- Use namespaces combined with RBAC to enforce least-privilege access for different teams.
- Set resource requests and limits on every container to prevent noisy-neighbour problems.
- Use Pod Disruption Budgets to maintain minimum availability during node upgrades.
- Store Helm values files in Git; never run helm upgrade with inline --set flags in production.
- Enable cluster autoscaler so the node group scales automatically with pod demand.

Challenges and Solutions

Challenge

Persistent storage management is complex across availability zones.

Solution

Use topology-aware volume provisioning; ensure StatefulSets schedule pods and volumes in the same AZ.

Challenge

Rolling updates can cause brief downtime if readiness probes are not configured.

Solution

Define accurate readiness and liveness probes; set minReadySeconds and maxUnavailable appropriately.

Challenge

RBAC complexity increases as the team grows.

Solution

Use Kubernetes RBAC combined with AWS IAM access entries; document roles clearly and review permissions quarterly.

Project 4: Docker Containerisation of a Multi-Tier Application

Project 4 of 50 • Containerisation

Description

Containerise a three-tier web application (React frontend, Node.js API, MySQL database) using Docker and Docker Compose. Each service runs in its own container, communicates over a defined network, and is configured via environment variables, making the stack portable and easy to run locally or in any cloud.

Use Case

Development teams that need identical environments across developer laptops, CI systems, and production. Solves the classic 'works on my machine' problem and speeds up onboarding of new developers from days to minutes.

Tools and Technologies

Docker, Docker Compose, Docker Hub / ECR, multi-stage Dockerfiles, .dockerignore, Trivy (image scanning), GitHub Actions

Implementation Steps

1. Write multi-stage Dockerfiles for frontend (build stage with Node, serve stage with nginx) and backend.
2. Create a .dockerignore file to exclude node_modules, .git, and test files from the build context.
3. Define a docker-compose.yml with services, named networks, and named volumes.
4. Configure environment variables via .env files; never hardcode credentials in Dockerfiles.
5. Set up a healthcheck instruction in each Dockerfile so Docker and Compose know when a service is ready.
6. Build and push images to a container registry (ECR or Docker Hub) with semantic version tags.
7. Integrate Trivy image scanning into the CI pipeline; fail on HIGH/CRITICAL vulnerabilities.
8. Document the local development workflow in the README: clone, copy .env.example, run docker compose up.

Best Practices

- Use specific base image tags (e.g., node:20-alpine) never 'latest' to ensure reproducible builds.
- Run containers as a non-root user by adding a USER instruction in the Dockerfile.
- Keep images small by using Alpine variants and removing build dependencies in multi-stage builds.
- Scan images regularly, not just at build time, since new CVEs emerge after an image is published.
- Use Docker secrets or environment injection at runtime rather than baking secrets into images.

Challenges and Solutions

Challenge

Database containers lose data on restart.

Solution

Mount a named volume to the database data directory; never use bind mounts for database storage in production.

Challenge

Image sizes balloon making pulls slow.

Solution

Use multi-stage builds, minimise layers, and clean package caches in the same RUN instruction.

Challenge

Service startup order causes connection errors.

Solution

Use depends_on with condition: service_healthy and implement retry logic in application startup code.

Project 5: GitOps Workflow with ArgoCD

Project 5 of 50 • GitOps

Description

Implement a GitOps deployment model where the desired state of Kubernetes applications is stored in a Git repository and ArgoCD continuously reconciles the live cluster state with the declared state. Any change to the cluster must go through a pull request, providing a complete audit trail of every deployment.

Use Case

Platform teams managing multiple Kubernetes clusters across environments. GitOps eliminates manual kubectl apply commands, prevents configuration drift, and gives every deployment a traceable PR link, making rollbacks as simple as reverting a commit.

Tools and Technologies

ArgoCD, Helm, Kustomize, Git (GitHub/GitLab), Sealed Secrets or External Secrets Operator, GitHub Actions (for image updates), Renovate Bot

Implementation Steps

1. Create a dedicated GitOps repository separate from application code.
2. Structure the repo: apps/ (ArgoCD Application manifests), charts/ (Helm values), overlays/ (Kustomize patches per environment).
3. Install ArgoCD in the cluster using the official Helm chart; expose the UI via Ingress.
4. Create ArgoCD Application resources pointing to the GitOps repo paths.
5. Configure automated sync with self-heal: ArgoCD corrects any manual cluster change within seconds.
6. Set up Sealed Secrets or External Secrets Operator to manage Kubernetes secrets safely in Git.
7. Automate image tag updates in the GitOps repo using GitHub Actions triggered by new image pushes.
8. Configure Slack notifications for sync success, failure, and out-of-sync states.

Best Practices

- Use separate Git branches or directories for each environment; promote changes from dev → staging → prod via PRs.
- Enable ApplicationSet to generate Application resources automatically for every service following a naming convention.
- Restrict direct cluster access; require all changes to go through the GitOps repo.
- Use Renovate or Dependabot to keep Helm chart dependencies and container image versions up to date.
- Implement a sync window policy to prevent auto-sync during peak business hours for production.

Challenges and Solutions

Challenge

Secrets cannot be stored as plaintext in Git.

Solution

Use Sealed Secrets to encrypt secrets; only the controller in the cluster can decrypt them.

Challenge

Merge conflicts in values files when multiple teams update simultaneously.

Solution

Use Kustomize overlays to keep team-specific patches separate, reducing conflict surface area.

Challenge

Drift between what teams test locally and what is in Git.

Solution

Enforce policy that local testing uses mirrored Git values; use pre-commit hooks to validate manifest syntax.

Project 6: Monitoring and Alerting with Prometheus and Grafana

Project 6 of 50 • Observability

Description

Deploy a full observability stack using Prometheus for metrics collection and Grafana for dashboards. Monitor application performance, infrastructure health, and business KPIs in real time. Configure alert rules that page the on-call engineer when SLOs are breached.

Use Case

Any team running production workloads that need visibility into system health. Replaces guesswork with data — teams can pinpoint the exact cause of slowdowns, proactively fix resource exhaustion, and prove SLA compliance to stakeholders.

Tools and Technologies

Prometheus, Grafana, Alertmanager, node_exporter, kube-state-metrics, Pushgateway, PagerDuty / Opsgenie, Loki (optional logs), Helm (kube-prometheus-stack)

Implementation Steps

1. Deploy the kube-prometheus-stack Helm chart which installs Prometheus, Alertmanager, Grafana, and exporters in one go.
2. Configure ServiceMonitors for application services so Prometheus scrapes custom metrics automatically.
3. Import community Grafana dashboards for Kubernetes cluster overview, node health, and common databases.
4. Define PrometheusRule resources for alert conditions: CPU > 80% for 5 minutes, pod restarts > 3 in 10 minutes, error rate > 1%.
5. Configure Alertmanager routing: critical alerts go to PagerDuty; warnings go to Slack.
6. Instrument application code with Prometheus client libraries to expose request rates, latencies, and error counts.
7. Create SLO dashboards showing error budget burn rates over 30-day rolling windows.
8. Set up Grafana provisioning so dashboards and data sources are version-controlled in Git.

Best Practices

- Follow the RED method (Rate, Errors, Duration) for service dashboards and USE (Utilisation, Saturation, Errors) for infrastructure.
- Set alert thresholds based on historical data, not gut feeling; start lenient and tighten over time.
- Avoid alert fatigue by ensuring every alert is actionable and has a clear runbook.
- Retain raw metrics for 15 days and aggregated metrics for 1 year for capacity planning.
- Test alerts regularly by intentionally triggering conditions in a staging environment.

Challenges and Solutions

Challenge	Solution
-----------	----------

Prometheus high cardinality metrics consume excessive memory.

Audit label cardinalities; avoid user IDs or request IDs as metric labels. Use recording rules to pre-aggregate.

Challenge

Alert noise overwhelms the on-call team.

Solution

Implement alert grouping, inhibition rules, and a noise budget review in each sprint.

Challenge

Grafana dashboards go stale as services evolve.

Solution

Store dashboards as JSON in Git; use CI to validate and provision them automatically on merge.

Project 7: Centralised Log Management with ELK Stack

Project 7 of 50 • Observability

Description

Build a centralised logging platform using Elasticsearch, Logstash, and Kibana (ELK). Aggregate logs from hundreds of services and servers, parse them into structured fields, and enable full-text search and visualisation. Engineers can trace a single request across multiple services in seconds.

Use Case

Operations and support teams troubleshooting production incidents. Instead of SSH-ing into individual servers and running grep commands, engineers search and filter across all logs in Kibana, reducing mean time to resolution from hours to minutes.

Tools and Technologies

Elasticsearch, Logstash, Kibana, Filebeat, Metricbeat, Fluentd (alternative to Logstash), AWS OpenSearch, Index Lifecycle Management, Watcher

Implementation Steps

1. Deploy a 3-node Elasticsearch cluster with dedicated master, data, and coordinating nodes.
2. Configure Kibana pointing to the Elasticsearch cluster; enable HTTPS and role-based access.
3. Deploy Filebeat as a DaemonSet on Kubernetes to ship container logs to Logstash.
4. Write Logstash pipelines to parse log formats (JSON, Apache, custom patterns) using grok filters.
5. Create Elasticsearch index templates defining field mappings and shard settings.
6. Configure Index Lifecycle Management (ILM) policies: hot → warm → cold → delete phases.
7. Build Kibana dashboards for application errors, slow queries, and security events.
8. Set up Watcher alerts for error rate spikes and ship them to Slack or PagerDuty.

Best Practices

- Parse logs into structured fields at ingest time; searching on raw text is slow and brittle.
- Enforce a common log schema across all services: timestamp, level, service, traceId, message.
- Use ILM to automatically roll over and delete old indices; unchecked growth will fill disks.
- Limit Kibana access by role: developers see their service logs; security team sees audit logs.
- Test Logstash pipelines locally with sample log files before deploying to production.

Challenges and Solutions

Challenge

Elasticsearch cluster becomes a single point of failure.

Solution

Run at least 3 master-eligible nodes and replicate all indices. Use AWS OpenSearch for managed HA.

Challenge

Log volume spikes overwhelm Logstash.

Solution

Add a Kafka or Redis buffer between Filebeat and Logstash to absorb bursts without dropping messages.

Challenge

PII in logs creates compliance risk.

Solution

Use Logstash mutate filters to mask or drop sensitive fields before indexing.

Project 8: Secrets Management with HashiCorp Vault

Project 8 of 50 • Security & DevSecOps

Description

Deploy HashiCorp Vault to centrally manage secrets, API keys, database credentials, and TLS certificates. Applications retrieve secrets dynamically at runtime instead of reading static files, and short-lived credentials reduce the blast radius of any security breach.

Use Case

Organisations handling regulated data (PCI DSS, HIPAA, SOC 2) that require auditable secret access logs and automatic credential rotation. Also valuable for teams tired of secrets leaking in Git history or environment variable dumps.

Tools and Technologies

HashiCorp Vault, Vault Agent, Vault Secrets Operator (Kubernetes), AWS KMS (auto-unseal), Consul (HA backend), AppRole / Kubernetes auth, Terraform Vault provider

Implementation Steps

1. Deploy Vault in HA mode using Raft integrated storage on Kubernetes via the official Helm chart.
2. Configure AWS KMS auto-unseal so Vault unseals automatically after pod restarts.
3. Enable the KV v2 secrets engine for static secrets; enable the database secrets engine for dynamic credentials.
4. Configure the Kubernetes auth method so pods authenticate using their service account JWT.
5. Write Vault policies granting each application access only to the secrets it needs (least privilege).
6. Deploy the Vault Secrets Operator to sync Vault secrets into Kubernetes Secrets automatically.
7. Enable the PKI secrets engine to issue short-lived TLS certificates for internal services.
8. Configure audit logging to ship to the ELK stack for compliance reporting.

Best Practices

- Never store the unseal keys or root token — use KMS auto-unseal and revoke the root token after initial setup.
- Grant applications the minimum Vault policy required; scope policies to specific paths.
- Rotate dynamic database credentials every hour; applications should gracefully handle credential rotation.
- Monitor Vault audit logs for unusual access patterns as part of your SIEM strategy.
- Back up Vault snapshots regularly and test the restore process quarterly.

Challenges and Solutions

Challenge

Applications not designed to reload secrets break when Vault credentials expire.

Solution

	Use Vault Agent sidecar or the Vault Secrets Operator to refresh secrets in Kubernetes Secrets; application reads the file, not Vault directly.
Challenge Vault becomes unavailable during maintenance, blocking all deployments.	Solution Use HA mode with 3 nodes and configure application-level retry logic with exponential backoff.
Challenge Onboarding teams to Vault requires significant policy writing.	Solution Build a self-service portal backed by Terraform that teams use to request secret paths; pull requests trigger policy updates.

Project 9: Blue-Green Deployments on AWS

Project 9 of 50 • Deployment Strategies

Description

Implement blue-green deployments where two identical production environments (blue and green) run side by side. New versions deploy to the idle environment, get tested, and traffic switches instantly via a load balancer change. Rollbacks take seconds, not minutes.

Use Case

E-commerce and SaaS companies where downtime directly costs revenue. Blue-green enables zero-downtime releases and instant rollbacks, making it ideal for high-stakes deployments or organisations with strict SLA commitments.

Tools and Technologies

AWS CodeDeploy, AWS Elastic Load Balancer (ALB), AWS Auto Scaling Groups, Route 53, GitHub Actions, Terraform, CloudWatch

Implementation Steps

1. Provision two identical Auto Scaling Groups (blue and green) behind a single ALB with two target groups.
2. Write a CodeDeploy deployment group configured for blue-green deployment type.
3. Create a GitHub Actions workflow that builds and pushes a new AMI or container image and triggers CodeDeploy.
4. Configure CodeDeploy to deploy to the idle (green) group, run health checks, and wait for approval.
5. After approval, shift 10% of traffic to green, monitor error rates for 5 minutes, then shift 100%.
6. Keep the blue environment running for 30 minutes as an instant rollback option.
7. Set up CloudWatch alarms that trigger an automatic rollback if error rate exceeds 1% during cutover.
8. Automate post-deployment cleanup: terminate old blue instances after the grace period.

Best Practices

- Ensure database migrations are backward compatible — both environments must run against the same schema during the transition.
- Use feature flags to separate code deployment from feature release; deploy dark, enable incrementally.
- Test the green environment with synthetic traffic before switching real users.
- Tag resources clearly with environment colour so cost dashboards show the cost of keeping the idle stack.
- Automate the rollback trigger so on-call engineers do not need to act within a narrow window.

Challenges and Solutions

Challenge

Database schema changes are incompatible between blue and green.

Solution

Apply expand-contract migrations: add new columns before deployment, drop old columns after full traffic cutover.

Challenge

Running two full environments doubles infrastructure cost.

Solution

Use scaled-down idle environments (1 instance) and scale out only when a deployment is in progress.

Challenge

Sticky sessions break after traffic shift.

Solution

Use distributed session storage (Redis/ElastiCache) so users can be served by either environment without losing state.

Project 10: Canary Deployments with Flagger and Istio

Project 10 of 50 • Deployment Strategies

Description

Automate canary releases using Flagger and Istio service mesh. New application versions receive a small percentage of traffic, and Flagger automatically analyses metrics. If error rates or latency thresholds are exceeded, the release rolls back automatically without human intervention.

Use Case

High-traffic platforms that cannot afford big-bang releases. Canary deployments allow teams to validate new versions on real production traffic with minimal user impact, especially useful when A/B testing performance improvements or risky algorithmic changes.

Tools and Technologies

Kubernetes, Istio, Flagger, Prometheus, Grafana, Helm, GitHub Actions, Slack

Implementation Steps

1. Install Istio on the Kubernetes cluster using the `istioctl install` command with the default profile.
2. Enable Istio sidecar injection on the application namespace.
3. Install Flagger using Helm with the Istio provider option.
4. Create a Canary custom resource specifying: `canaryAnalysis` thresholds (max latency, error rate), traffic step weight (10%), and interval (1 minute).
5. Configure a `MetricTemplate` in Flagger pointing to Prometheus queries for request success rate and P99 latency.
6. Trigger a canary by updating the Deployment image tag; Flagger takes over and manages traffic shifting.
7. Configure Flagger Slack alerting for promotion, rollback, and analysis events.
8. Integrate canary promotion into the CI/CD pipeline: the pipeline updates the image tag and watches Flagger status.

Best Practices

- Define canary analysis metrics that directly reflect user experience: error rate and latency, not just CPU.
- Start with a 5% canary weight and use small step increments (5–10%) with 2-minute analysis intervals.
- Run at least 3 successful analysis iterations before considering a canary healthy.
- Use a dedicated canary namespace for isolation so analysis traffic does not interfere with primary metrics.
- Document the rollback procedure even though it is automated — engineers must understand what to look for.

Challenges and Solutions

Challenge

Insufficient traffic volume makes metric analysis statistically unreliable.

Solution

Set a minimum request threshold in the canary analysis config; only promote when enough samples exist.

Challenge

Istio sidecar injection increases pod startup time.

Solution

Tune readiness probe delays and set appropriate termination grace periods to account for sidecar initialisation.

Challenge

Canary analysis fires false positives during temporary traffic dips.

Solution

Use percentile metrics (P95/P99) rather than averages; add a minimum-requests-per-interval threshold.

Project 11: DevSecOps Pipeline with SAST, DAST, and SCA

Project 11 of 50 • Security & DevSecOps

Description

Embed security scanning directly into the CI/CD pipeline using Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), and Software Composition Analysis (SCA). Security findings are treated like build failures — the pipeline stops until issues are resolved or risk is accepted.

Use Case

Development teams subject to security audits, PCI DSS, or SOC 2 requirements. Shifting security left reduces the cost of fixing vulnerabilities by up to 100x compared to finding them post-production, and automates the evidence collection needed for compliance reports.

Tools and Technologies

SonarQube (SAST), OWASP ZAP (DAST), Trivy / Snyk (SCA), Gitleaks (secret scanning), Checkov (IaC scanning), GitHub Actions, Dependency-Check, Semgrep

Implementation Steps

1. Add Gitleaks to pre-commit hooks and the pipeline first stage to block secret commits.
2. Configure SonarQube with quality gates: fail if new blocker or critical issues are introduced.
3. Run Trivy filesystem scan on the repository to detect vulnerable dependencies (SCA).
4. Build the Docker image and run Trivy image scan; fail on HIGH/CRITICAL CVEs with available fixes.
5. Run Checkov against all Terraform and Kubernetes manifests for IaC misconfigurations.
6. Deploy the application to an ephemeral test environment and run OWASP ZAP baseline scan.
7. Parse all tool outputs into a unified SARIF format; upload to GitHub Security tab.
8. Generate a Software Bill of Materials (SBOM) with Trivy and attach it to the release artefact.

Best Practices

- Enforce security gates as hard failures in the pipeline, not advisory warnings that get ignored.
- Establish a vulnerability management SLA: Critical = 24 hours, High = 7 days, Medium = 30 days.
- Use suppression files to document accepted risks with justification and expiry dates.
- Run DAST against a staging environment that mirrors production data shapes but uses anonymised data.
- Review and tune SAST rules quarterly to reduce false positives and maintain developer trust.

Challenges and Solutions

Challenge

Security scans add 10–15 minutes to the pipeline, slowing developer feedback.

Solution

Run DAST in a parallel pipeline stage; run SAST only on changed files using incremental analysis.

Challenge

High false positive rates cause developers to ignore scan results.

Solution

Tune SonarQube rules per language; suppress known false positives with documented justification.

Challenge

Fixing all found vulnerabilities is overwhelming for the first run.

Solution

Establish a baseline and block only new issues introduced after a cutoff date; backlog old issues for triage.

Project 12: Multi-Cloud Infrastructure Management

Project 12 of 50 • Infrastructure Automation

Description

Design and manage infrastructure spanning AWS, Azure, and GCP using Terraform and a unified CI/CD pipeline. Workloads run in the optimal cloud based on cost, compliance, and feature availability, while a single set of tools manages them all.

Use Case

Enterprises that use multiple clouds due to acquisitions, regulatory requirements (data residency), or negotiated pricing. A multi-cloud approach also prevents vendor lock-in and improves resilience — if one provider has an outage, workloads can shift.

Tools and Technologies

Terraform (with AWS, Azure, GCP providers), Terragrunt, Atlantis, GitHub Actions, Infracost, Cloud Custodian, Datadog (multi-cloud monitoring)

Implementation Steps

1. Structure Terraform in a monorepo with cloud-specific directories (aws/, azure/, gcp/) sharing common modules.
2. Use Terragrunt to DRY up repeated backend and provider configurations across environments.
3. Configure provider credentials using federated identity (OIDC) for each cloud in GitHub Actions.
4. Deploy Atlantis as a Terraform automation server; all plans and applies happen via PR comments.
5. Integrate Infracost into the PR pipeline to show monthly cost estimates for infrastructure changes.
6. Define cross-cloud networking: AWS Transit Gateway, Azure VPN Gateway, and GCP Cloud Interconnect.
7. Set up a single Datadog account with AWS, Azure, and GCP integrations for unified monitoring.
8. Use Cloud Custodian to enforce tagging, cost controls, and security policies across all clouds.

Best Practices

- Avoid cloud-specific abstractions in shared modules; build provider-agnostic interfaces.
- Consolidate billing: use AWS Organisations, Azure Management Groups, and GCP Billing Accounts with a single cost dashboard.
- Document which workloads run in which cloud and why; avoid accidental spread driven by habit.
- Test disaster recovery procedures by simulating a cloud region failure and failing over to another cloud.
- Use a cloud-agnostic load balancer (Cloudflare, AWS Global Accelerator) to route traffic between clouds.

Challenges and Solutions

Challenge

Each cloud has different IAM models, making unified access control difficult.

Solution

Map cloud-specific IAM concepts to a common model; use a central identity provider (Okta, Azure AD) with OIDC federation.

Challenge

Latency and cost of cross-cloud data transfer is significant.

Solution

Minimise cross-cloud calls; co-locate tightly coupled services within the same cloud. Use CDN for static assets.

Challenge

Terraform state management across three clouds is complex.

Solution

Use a separate state bucket per cloud with consistent naming; use Terragrunt to manage remote state config automatically.

Project 13: Automated Database Schema Migration Pipeline

Project 13 of 50 • Database DevOps

Description

Automate database schema changes using Liquibase or Flyway as part of the CI/CD pipeline. Every schema change is version-controlled, peer-reviewed, tested in staging, and applied to production automatically — eliminating manual DBA involvement for routine migrations and reducing errors.

Use Case

Development teams with frequent schema changes and a desire to eliminate manual runbook steps during deployments. Critical for teams running microservices where each service owns its database schema and needs independent migration capability.

Tools and Technologies

Flyway or Liquibase, GitHub Actions, PostgreSQL/MySQL/RDS, Docker (for local testing), Testcontainers (for integration tests), Slack notifications

Implementation Steps

1. Add Flyway to the project dependencies; create db/migration/ directory for SQL migration files.
2. Follow the naming convention: V{version}__{description}.sql (e.g., V003__add_user_email_index.sql).
3. Write repeatable migrations (R__ prefix) for views and stored procedures.
4. Add a Testcontainers-based integration test that spins up a real database and runs all migrations.
5. In the CI pipeline, run flyway validate before applying to catch checksum mismatches.
6. In the staging pipeline stage, run flyway migrate against the staging database.
7. Add a manual approval gate before production migration; notify the DBA team via Slack.
8. Configure Flyway out-of-order protection; alert if someone has applied a migration out of sequence.

Best Practices

- Never edit an applied migration file; always create a new migration to fix mistakes.
- Write every migration to be idempotent where possible (CREATE TABLE IF NOT EXISTS).
- Pair destructive migrations (DROP COLUMN) with a feature flag to decouple code and schema changes.
- Include rollback SQL in comments within the migration file; not all databases support automatic rollback of DDL.
- Test migrations against a production-sized dataset in staging to catch performance issues before they hit production.

Challenges and Solutions

Challenge

Long-running migrations lock tables and cause application timeouts.

Solution

Use online schema change tools (pt-online-schema-change, gh-ost) for large table alterations that avoid table locks.

Challenge

Merge conflicts in migration files when multiple developers add migrations simultaneously.

Solution

Use timestamp-based version prefixes instead of sequential numbers to eliminate ordering conflicts.

Challenge

Rollbacks for data migrations are extremely difficult.

Solution

Separate schema migrations (DDL) from data migrations (DML); keep data migrations reversible and run them as separate, off-hours jobs.

Project 14: Service Mesh Implementation with Istio

Project 14 of 50 • Networking & Service Mesh

Description

Deploy Istio service mesh across a Kubernetes cluster to add mutual TLS encryption, advanced traffic management, observability, and rate limiting to all service-to-service communication without changing any application code. The infrastructure team manages cross-cutting network concerns centrally.

Use Case

Organisations with strict compliance requirements for encrypted service communication, or teams running complex microservice architectures that need fine-grained traffic control, circuit breaking, and distributed tracing without burdening every application team with these concerns.

Tools and Technologies

Istio, Envoy (sidecar proxy), Kiali (service graph), Jaeger (distributed tracing), Prometheus, Grafana, Cert-Manager, Kubernetes

Implementation Steps

1. Install Istio using `istioctl` with the default profile and enable the Prometheus, Grafana, Kiali, and Jaeger add-ons.
2. Enable automatic sidecar injection on application namespaces with the `istio-injection=enabled` label.
3. Create `PeerAuthentication` resources to enforce STRICT mTLS mode within the mesh.
4. Write `VirtualService` and `DestinationRule` resources to configure load balancing, retries, and timeouts per service.
5. Implement circuit breaking with outlier detection in `DestinationRules`.
6. Configure `AuthorizationPolicy` resources to allow only specific services to communicate with each other.
7. Set up ingress Gateway to terminate external TLS and route traffic to in-mesh services.
8. Explore Kiali to visualise the service graph and identify unhealthy service relationships.

Best Practices

- Enforce STRICT mTLS mode; never leave services in PERMISSIVE mode in production.
- Apply fine-grained `AuthorizationPolicy` to every service — start with `deny-all` and add allows explicitly.
- Set sane default timeouts and retry policies at the mesh level; override per service where needed.
- Monitor Envoy sidecar resource consumption; sidecars add CPU and memory overhead per pod.
- Upgrade Istio incrementally; canary-upgrade the control plane before the data plane.

Challenges and Solutions

Challenge

Sidecar injection breaks some legacy applications that do not handle connection interruptions.

Solution

Use traffic interception exclusions for specific ports and protocols; incrementally migrate services to the mesh.

Challenge

Debugging mTLS handshake failures is non-trivial.

Solution

Use `istioctl proxy-status` and `istioctl proxy-config` to inspect Envoy configuration and certificate chain.

Challenge

Complexity of YAML configuration grows rapidly.

Solution

Use GitOps and Kustomize to manage Istio configuration; build a library of reusable policy templates.

Project 15: Cost Optimisation Dashboard for Cloud Spend

Project 15 of 50 • FinOps

Description

Build a real-time cloud cost visibility and optimisation platform using AWS Cost Explorer, Grafana, and automation scripts. Identify waste, right-size resources, and implement showback/chargeback reporting so each team sees the cost of their workloads.

Use Case

FinOps teams and engineering managers trying to control cloud spend as organisations scale. Cloud costs often grow 3x faster than the business without deliberate governance. This project provides the visibility and automation needed to hold the line.

Tools and Technologies

AWS Cost Explorer API, AWS Budgets, AWS Trusted Advisor, Grafana, Prometheus (cloud_cost_exporter), Infracost, Kubecost, CloudHealth, Python/Lambda for automation

Implementation Steps

1. Enable AWS Cost Explorer and activate cost allocation tags for all resources (team, environment, service).
2. Write a Lambda function that queries Cost Explorer daily and stores results in a DynamoDB table.
3. Build a Grafana dashboard pulling from DynamoDB showing daily/monthly spend by team and service.
4. Set up AWS Budgets alerts: notify at 75% and 100% of monthly budget per team.
5. Deploy Kubecost in the Kubernetes cluster to show per-namespace and per-deployment cost.
6. Schedule a weekly Lambda that identifies idle resources: unattached EBS volumes, stopped instances, unused load balancers.
7. Automate right-sizing recommendations: use Trusted Advisor and AWS Compute Optimiser API to generate tickets.
8. Integrate Infracost into the CI pipeline so PRs show the cost change of infrastructure modifications.

Best Practices

- Enforce a tagging policy via AWS Config rules; untagged resources are automatically flagged.
- Review Reserved Instance and Savings Plan coverage monthly; commit to savings plans for predictable workloads.
- Delete development environments over weekends using Instance Scheduler to cut costs by 30–40%.
- Set up anomaly detection in AWS Cost Anomaly Detection to catch sudden spend spikes immediately.
- Review and right-size instances every quarter based on CloudWatch utilisation data.

Challenges and Solutions

Challenge	Solution
-----------	----------

Cost allocation tags are inconsistently applied across teams.

Enforce tags at resource creation using SCPs (Service Control Policies) and AWS Config; block deployments from the CI pipeline if required tags are missing.

Challenge

Chargebacks create political conflict between teams.

Solution

Start with showback (visibility, no billing impact) before implementing chargeback; educate teams on the data first.

Challenge

Spot Instance interruptions disrupt batch workloads.

Solution

Design workloads to checkpoint state; use Spot Interruption Notices (2-minute warning) to drain gracefully.

Project 16: Automated Incident Response with PagerDuty and Runbooks

Project 16 of 50 • Incident Management

Description

Build an automated incident response system that detects anomalies, creates incident tickets, pages the correct on-call engineer, and executes automated runbooks to resolve common issues without human intervention. Mean time to resolution drops from hours to minutes.

Use Case

Operations teams running 24/7 services who want to reduce on-call burden and ensure consistent incident handling. Automation handles the first 70% of common incidents (disk full, pod crash loop, certificate expiry) so engineers focus on complex problems.

Tools and Technologies

PagerDuty, Prometheus Alertmanager, AWS Lambda, Jira/ServiceNow, Slack, Ansible/Python runbooks, AWS Systems Manager Run Command, Opsgenie

Implementation Steps

1. Define incident severity levels (P1–P4) with clear criteria and expected response times.
2. Configure Alertmanager to route alerts to PagerDuty based on severity and service.
3. Write automated runbooks as Lambda functions: restart crashed pods, clear disk space, renew certificates.
4. Configure PagerDuty Event Rules to trigger the appropriate Lambda runbook for known alert types.
5. Integrate Jira: PagerDuty auto-creates Jira tickets for P1/P2 incidents with relevant context.
6. Build a Slack bot that posts incident channels, @-mentions the on-call, and provides runbook links.
7. Implement a post-incident review (PIR) template in Confluence that is auto-created 24 hours after a P1.
8. Track MTTA (mean time to acknowledge) and MTTR (mean time to resolve) in a Grafana dashboard.

Best Practices

- Every alert must have a runbook link in its annotations; on-call engineers should never have to search for context.
- Practice incident response with GameDay exercises quarterly — simulate failures in staging and run the response.
- Review on-call burden monthly: more than 2 pages per shift indicates insufficient automation or alert tuning.
- Publish incident post-mortems publicly internally; blameless culture accelerates learning.
- Automate toil: if an engineer performs the same manual remediation 3 times, it should be automated.

Challenges and Solutions

Challenge

Automated runbooks take incorrect actions and worsen the incident.

Solution

Start with information-gathering runbooks (collect logs, metrics snapshots) before implementing automated remediation. Test all runbooks in staging.

Challenge

Alert grouping in PagerDuty creates noise during large incidents.

Solution

Configure intelligent alert grouping and suppression; use PagerDuty's correlation to group related alerts into one incident.

Challenge

On-call rotation burnout and reluctance to join rotation.

Solution

Involve engineers in alert tuning; give them authority to silence noisy alerts and track noise metrics publicly.

Project 17: Container Image Hardening and Vulnerability Management

Project 17 of 50 • Security & DevSecOps

Description

Establish a container image hardening programme that produces minimal, secure base images and operates a continuous vulnerability management pipeline. All production images are built from approved hardened bases, scanned on every build and on a regular schedule, and automatically rebuilt when new patches are available.

Use Case

Security and platform teams responsible for the security of containerised workloads. Hardened images reduce the attack surface and the number of CVEs teams need to triage, leading to cleaner scan results and faster time-to-compliance.

Tools and Technologies

Docker, Trivy, Gype, Dockle, Docker Bench for Security, AWS ECR (image scanning), Cosign (image signing), Renovate Bot, GitHub Actions, Distroless images

Implementation Steps

1. Create hardened base images based on distroless or Alpine; remove shells, package managers, and unused tools.
2. Use multi-stage builds so only runtime artefacts land in the final image layer.
3. Add a USER instruction to run as a non-root user; set read-only root filesystem where possible.
4. Scan all images with Trivy and Gype in the CI pipeline; fail on HIGH/CRITICAL with available fixes.
5. Run Dockle to check image best practices (e.g., no SUID binaries, no default passwords).
6. Sign images with Cosign and verify signatures in admission controllers (Kyverno/OPA Gatekeeper) before scheduling.
7. Configure ECR to rescan images on a daily schedule; set up SNS notifications for new critical CVEs.
8. Use Renovate Bot to automatically open PRs updating base image digests when patched versions are published.

Best Practices

- Pin base images by digest (sha256), not tag, to prevent silent changes from upstream updates.
- Maintain an internal base image registry with approved, continuously patched images; teams pull from here, not Docker Hub.
- Never install debugging tools in production images; use ephemeral debug containers (kubectl debug) for investigation.
- Track the mean time to patch critical CVEs as a security KPI; target less than 5 business days.
- Enforce an admission webhook that rejects unsigned images and images with unacceptable scan grades.

Challenges and Solutions

Challenge

Distroless images lack a shell, making debugging difficult.

Solution

Use kubectl debug with a sidecar debug image for runtime investigation; document this process for the team.

Challenge

Renovate PRs for base image updates are too frequent and developers ignore them.

Solution

Group base image update PRs by schedule (weekly); auto-merge if CI passes, reducing review fatigue.

Challenge

Scanners report false positives for OS packages that are not actually installed.

Solution

Cross-reference SBOM data with scanner output; suppress false positives with documented evidence.

Project 18: Platform Engineering: Internal Developer Platform (IDP)

Project 18 of 50 • Platform Engineering

Description

Build an Internal Developer Platform (IDP) using Backstage that gives developers a single place to create services, manage infrastructure, view documentation, and monitor deployments. The platform team operates the golden paths; developers choose from pre-approved, production-ready templates and focus on business logic.

Use Case

Engineering organisations with 50+ developers where the cognitive overhead of navigating cloud infrastructure, CI/CD tools, and security scanners slows feature delivery. An IDP reduces the time from idea to first production deployment from days to hours.

Tools and Technologies

Backstage, GitHub, Terraform Cloud, Kubernetes, ArgoCD, PagerDuty, GitHub Actions, Crossplane, Port (alternative IDP), Catalog plugins

Implementation Steps

1. Deploy Backstage with a PostgreSQL backend; configure GitHub OAuth for authentication.
2. Populate the software catalog by importing all repositories and their catalog-info.yaml files.
3. Create Software Templates for common service types: REST API, event-driven service, frontend SPA.
4. Wire templates to GitHub Actions workflows that provision repositories, CI pipelines, and ArgoCD applications automatically.
5. Install Backstage plugins: Kubernetes (pod status in catalog), PagerDuty (on-call info), GitHub Actions (pipeline status), TechDocs.
6. Write TechDocs documentation that renders markdown from each service's docs/ directory in Backstage.
7. Integrate Crossplane to allow teams to provision cloud resources (databases, queues) via Backstage with guardrails.
8. Track golden path adoption: measure % of services created via templates vs. manually.

Best Practices

- Start small: launch with one golden path and one integration, then expand based on developer feedback.
- Treat the IDP like a product — have a dedicated team, backlog, and regular user research sessions.
- Enforce catalog completeness: services missing owners, documentation, or runbooks are flagged in reports.
- Use Backstage as the single pane of glass; integrate it with every developer tool to avoid context switching.
- Measure time-to-production and developer satisfaction (DORA metrics + NPS) before and after IDP launch.

Challenges and Solutions

Challenge

Backstage requires significant initial setup and ongoing maintenance.

Solution

Use Roadie (managed Backstage) for smaller teams; build in-house only when you have dedicated platform engineering headcount.

Challenge

Keeping the software catalog up to date is a continuous effort.

Solution

Automate catalog entity creation via GitHub webhooks; require catalog-info.yaml as part of repo creation templates.

Challenge

Developer adoption is low if the platform adds friction instead of removing it.

Solution

Co-design workflows with developers; the platform must be faster and easier than the manual path from day one.

Project 19: Chaos Engineering with LitmusChaos

Project 19 of 50 • Reliability Engineering

Description

Implement a chaos engineering programme using LitmusChaos to proactively inject failures — pod kills, network delays, disk fill — into the staging and production environment in a controlled manner. Experiments validate that the system behaves gracefully under failure conditions before customers encounter real incidents.

Use Case

Teams building highly available services who want to validate their resilience claims. Moving from 'we hope it works' to 'we have run this failure scenario 50 times and know exactly how the system responds'. Essential before major traffic events (product launches, Black Friday).

Tools and Technologies

LitmusChaos, Kubernetes, Prometheus, Grafana (steady-state hypothesis dashboard), GitHub Actions, PagerDuty, Argo Workflows

Implementation Steps

1. Install LitmusChaos using Helm in a dedicated litmus namespace.
2. Define a steady-state hypothesis: the application serves requests at < 500ms P99 with < 0.1% error rate.
3. Create a ChaosExperiment for pod-delete targeting a specific deployment; set the blast radius to 1 pod at a time.
4. Create a ChaosSchedule to run the experiment every night in staging.
5. Monitor Prometheus during experiments; create a Grafana annotation to mark chaos start/end.
6. Run network-delay experiments (50ms added latency) to test timeout handling and circuit breakers.
7. Escalate to production: run pod-delete during low-traffic hours with PagerDuty alerting active.
8. Document every experiment result in a chaos journal in Confluence: hypothesis, method, result, action.

Best Practices

- Always define and validate the steady-state hypothesis before injecting chaos.
- Start with low blast radius in staging; graduate to production only after mastering staging experiments.
- Run experiments during business hours when the team is available to observe and respond.
- Treat a failed experiment (system degraded below hypothesis) as valuable information, not a failure.
- Build a chaos experiment library: re-run core experiments before every major release as a pre-flight check.

Challenges and Solutions

Challenge

Chaos experiments escape blast radius and impact other services.

Solution

Use namespace-scoped experiments and precise label selectors; always set appns and applabel carefully.

Challenge

Management is reluctant to approve production chaos experiments.

Solution

Build trust with a documented staging programme; share experiment results and the improvements they drove.

Challenge

False failures: monitoring triggers alerts during controlled experiments, training the team to ignore them.

Solution

Use LitmusChaos run annotations to suppress alerts during experiment windows in Alertmanager's time-based inhibition.

Project 20: Serverless CI/CD Pipeline on AWS with CodePipeline

Project 20 of 50 • Continuous Integration & Delivery

Description

Build an entirely serverless CI/CD pipeline using AWS CodePipeline, CodeBuild, and Lambda with no servers to manage. The pipeline compiles code, runs tests, scans for security issues, and deploys a serverless application (Lambda + API Gateway + DynamoDB) to multiple stages automatically.

Use Case

Small to medium teams building serverless applications who want a fully managed CI/CD experience without operating Jenkins servers. AWS CodePipeline integrates natively with all AWS services and is billed per pipeline execution, making it cost-effective for low-frequency pipelines.

Tools and Technologies

AWS CodePipeline, AWS CodeBuild, AWS CodeDeploy, AWS Lambda, AWS SAM, AWS CloudFormation, S3, ECR, IAM OIDC, Slack (via Lambda webhook)

Implementation Steps

1. Write the application using AWS SAM (Serverless Application Model); define Lambda functions, API Gateway, and DynamoDB in template.yaml.
2. Create a CodePipeline with four stages: Source (GitHub), Build (CodeBuild), Test (CodeBuild), Deploy.
3. Configure CodeBuild buildspec.yml: install SAM CLI, run unit tests, run sam build.
4. Use SAM pipeline to generate CodePipeline/CodeBuild configurations with built-in best practices.
5. Configure pipeline stages for dev, staging, and production with manual approval before production.
6. Enable CodeDeploy Linear10PercentEvery1Minute traffic shifting for Lambda deployments.
7. Add a rollback alarm: if error rate > 1% during traffic shift, CodeDeploy rolls back automatically.
8. Send pipeline state change events via EventBridge to a Lambda that posts Slack notifications.

Best Practices

- Use cross-account deployments: build artefacts in a tools account, deploy to separate dev/prod accounts.
- Store build artefacts in a versioned S3 bucket with lifecycle policies to expire old artefacts.
- Use IAM roles with least privilege for each CodeBuild project; do not use broad AdministratorAccess.
- Enable CodeBuild cache (S3 or local) to avoid re-downloading dependencies on every build.
- Tag all CodeBuild, CodePipeline, and deployment resources to track pipeline costs per service.

Challenges and Solutions

Challenge	Solution
Cold starts in Lambda add latency that fails integration tests.	Enable Provisioned Concurrency for critical functions during pipeline test stages; measure and set appropriate test timeouts.

Challenge

SAM template drift between developers on different machines.

Solution

Lock SAM CLI version in CodeBuild; provide a Docker-based local development environment with the same version.

Challenge

Cross-account permissions are complex to set up correctly.

Solution

Use AWS CDK Pipelines which handles cross-account assume-role configuration automatically.

Project 21: Container Registry and Image Lifecycle Management

Project 21 of 50 • Containerisation

Description

Establish a managed container registry using AWS ECR or Harbor with automated image scanning, lifecycle policies, cross-account replication, and immutable tags. The registry becomes the single trusted source of container images for all environments, with every image signed and every access logged.

Use Case

Platform teams managing 50+ container images across multiple services and environments. Without lifecycle management, registries fill with stale images, scanning alerts pile up, and teams lose track of which image version is running where.

Tools and Technologies

AWS ECR, Harbor, Docker, Cosign, Notary, Trivy, Renovate, GitHub Actions, AWS EventBridge, ECR lifecycle policies

Implementation Steps

1. Create ECR repositories with immutable tags enabled to prevent overwriting published images.
2. Enable ECR Enhanced Scanning (via Inspector) to continuously scan all stored images.
3. Configure ECR lifecycle policies: retain only the 20 most recent tagged images; delete untagged images after 7 days.
4. Set up cross-account replication: replicate production-approved images to the deployment account.
5. Integrate Cosign into the CI pipeline: sign images after successful scans and push signatures to ECR.
6. Configure an ECR pull-through cache for public registries (Docker Hub, Quay.io) to reduce egress costs.
7. Create an EventBridge rule that triggers a Lambda whenever ECR reports a new CRITICAL finding; Lambda creates a Jira ticket.
8. Implement SBOM generation per image and store SBOMs in S3 alongside release artefacts.

Best Practices

- Use image digest references in production deployments; tag references can be overwritten silently.
- Separate registries by trust level: internal dev, staging candidate, production approved.
- Review ECR Enhanced Scanning findings weekly; establish a team rotation for triage.
- Enforce image signing verification in Kubernetes admission controllers; unsigned images cannot run.
- Document the image promotion process: from dev registry → scan → sign → promote to production registry.

Challenges and Solutions

Challenge

Developers push large numbers of debug images that fill the registry and inflate costs.

Solution

	Enforce naming conventions and lifecycle policies per repository type. Auto-delete images without a semantic version tag after 3 days.
Challenge Vulnerability findings on base images affect dozens of application images.	Solution Rebuild all images from a patched base automatically using a Renovate or custom automation workflow triggered by base image updates.
Challenge Cross-account image access requires complex IAM policy management.	Solution Use ECR replication to push images to target accounts rather than relying on cross-account pull permissions.

Project 22: Distributed Tracing with Jaeger and OpenTelemetry

Project 22 of 50 • Observability

Description

Instrument a microservices application with OpenTelemetry to generate distributed traces and ship them to Jaeger. Engineers can visualise the complete journey of a single request across 10+ services, identify latency bottlenecks, and correlate traces with logs and metrics for rapid root cause analysis.

Use Case

Teams running microservices where a slow API response could originate from any of dozens of downstream services. Without distributed tracing, debugging requires piecing together logs from multiple services — a process that can take hours. Distributed tracing reduces this to seconds.

Tools and Technologies

OpenTelemetry SDK (Java/Node.js/Python), OpenTelemetry Collector, Jaeger, Elasticsearch (Jaeger backend), Grafana Tempo (alternative), Prometheus (metrics correlation), Kubernetes

Implementation Steps

1. Add the OpenTelemetry SDK to each service; configure automatic instrumentation for HTTP, gRPC, and database calls.
2. Deploy the OpenTelemetry Collector as a DaemonSet; configure receivers (OTLP), processors (batch, attribute), and exporters (Jaeger).
3. Set up Jaeger with Elasticsearch as the trace storage backend for scalable retention.
4. Configure trace sampling strategy: head-based sampling at 10%, with tail-based sampling for errors and slow requests.
5. Propagate trace context across service boundaries using W3C TraceContext headers.
6. Correlate trace IDs with log entries by injecting the current trace ID into structured logs.
7. Build a Grafana dashboard with a Jaeger trace link embedded alongside Prometheus metrics.
8. Create Jaeger service performance monitoring views showing P99 latency and error rates per operation.

Best Practices

- Use automatic instrumentation where possible; avoid hand-rolled span creation for common frameworks.
- Add business context as span attributes: order ID, user tier, feature flag state — this is invaluable during incidents.
- Tune sampling rates per service: high-traffic services need lower sampling rates to control storage costs.
- Ensure trace context propagation works across async message queues by injecting context into message headers.
- Retain full traces for at least 7 days; retain aggregated service graphs for 30 days.

Challenges and Solutions

Challenge	Solution
-----------	----------

Trace context is lost when crossing legacy services that do not propagate headers.

Inject a reverse proxy (Envoy) in front of legacy services to forward trace headers until they can be instrumented natively.

Challenge

Trace storage grows quickly at high sampling rates.

Solution

Implement tail-based sampling in the OpenTelemetry Collector: keep 100% of error traces and 5% of success traces.

Challenge

Developers resist adding instrumentation to application code.

Solution

Use OpenTelemetry auto-instrumentation agents that inject at startup without code changes; customise with configuration, not code.

Project 23: Automated Compliance as Code with Open Policy Agent

Project 23 of 50 • Security & DevSecOps

Description

Enforce compliance policies — naming conventions, required labels, forbidden capabilities, network policies — as code using Open Policy Agent (OPA) and Gatekeeper. Policies are evaluated at admission time in Kubernetes, blocking non-compliant resources before they are created, rather than finding violations after the fact.

Use Case

Enterprises with security and compliance teams that set policies but struggle to enforce them consistently across many development teams. OPA makes policies executable, auditable, and version-controlled, transforming compliance from a quarterly audit into continuous enforcement.

Tools and Technologies

OPA Gatekeeper, Rego (OPA policy language), Conftest, kubectl, GitHub Actions, Styra DAS (OPA management), kyverno (alternative), Terraform OPA integration

Implementation Steps

1. Install OPA Gatekeeper in the Kubernetes cluster using Helm.
2. Write a ConstraintTemplate in Rego for each policy: required labels, no latest image tags, read-only root filesystem required.
3. Create Constraint resources that apply each ConstraintTemplate to specific namespaces or resource types.
4. Test policies locally with Conftest before deploying to the cluster: `conftest test *.yaml --policy policies/`.
5. Add Conftest to the CI pipeline: validate all Kubernetes manifests against policies on every PR.
6. Run `kubectl get constraints` to list all active policies and their violation counts.
7. Set up a weekly OPA audit report: query Gatekeeper audit controller for current violations across the cluster.
8. Integrate OPA with Terraform using the OPA Terraform provider to enforce cloud resource policies.

Best Practices

- Start in dry-run mode to identify existing violations without blocking deployments; fix violations before enabling enforcement.
- Write human-readable error messages in constraint violation messages; developers must understand what to fix.
- Version control all ConstraintTemplates and Constraints in the GitOps repository.
- Review and update policies quarterly; remove policies that are no longer relevant to avoid compliance theatre.
- Use Gatekeeper audit mode to continuously scan existing resources for policy drift, not just at admission time.

Challenges and Solutions

Challenge Overly strict policies block legitimate use cases and erode developer trust.	Solution Build an exception request process: teams can get time-limited exclusions for specific namespaces with documented justification.
Challenge Writing Rego is difficult for team members unfamiliar with the language.	Solution Use the Styra OPA Playground for interactive testing; build a policy library and document each policy with examples.
Challenge Policy enforcement can break existing workloads during cluster upgrades.	Solution Run Gatekeeper in audit-only mode post-upgrade; review violations before switching back to enforcement mode.

Project 24: Feature Flag Management System

Project 24 of 50 • Deployment Strategies

Description

Implement a feature flag system using LaunchDarkly or Unleash to decouple code deployment from feature release. Features can be turned on or off instantly without a deployment, rolled out to specific user segments, and killed quickly if problems arise — all without touching the codebase.

Use Case

Product and engineering teams that want to release features progressively, test in production with specific beta users, or respond quickly to production incidents by disabling a problematic feature. Feature flags are essential infrastructure for continuous delivery at scale.

Tools and Technologies

LaunchDarkly, Unleash (open-source), Flagsmith, OpenFeature SDK, Redis (flag caching), GitHub Actions, Datadog (flag analytics), A/B testing platforms

Implementation Steps

1. Set up an Unleash server (or LaunchDarkly project) with environments: development, staging, production.
2. Define a feature flag naming convention: team-service-feature (e.g., payments-checkout-new-flow).
3. Integrate the Unleash client SDK into each application; implement a provider pattern to make testing easy.
4. Create a flag for a new feature with default state off; deploy the code to production behind the flag.
5. Enable the flag for the internal team first; gather feedback for one week.
6. Progressively roll out to 1%, 10%, 50%, and 100% of users, monitoring error rates at each step.
7. Set up a kill switch: a master flag that disables all new features simultaneously in emergencies.
8. Archive flags after features reach 100% rollout and the old code path is removed.

Best Practices

- Treat technical debt from flags seriously: schedule flag cleanup sprints; long-lived flags become maintenance nightmares.
- Use flag targeting for testing in production: enable for internal user emails before any external rollout.
- Cache flag evaluations locally in the application; never let a flag SDK call add latency to the hot path.
- Log flag evaluation decisions in structured logs; correlate flag state with incident timelines.
- Never use feature flags as long-term configuration management; they are for temporary states only.

Challenges and Solutions

Challenge

Flag explosion: hundreds of flags with unclear ownership and status.

Solution

Enforce a flag lifecycle: created → active → deprecated → removed. Assign owners and expiry dates at creation time.

Challenge

Flag evaluation adds latency to every API call.

Solution

Bootstrap flag state at application startup; re-evaluate in the background every 30 seconds rather than on each request.

Challenge

Testing code with flags in multiple states is complex.

Solution

Use the OpenFeature SDK's testing provider to set flag states deterministically in unit and integration tests.

Project 25: Automated Performance Testing Pipeline

Project 25 of 50 • Testing & Quality

Description

Build an automated performance testing pipeline that runs load tests on every release using k6 or Gatling. Performance budgets are enforced as pipeline gates — if a release regresses P99 response time by more than 10%, it fails the build and the change is not promoted to production.

Use Case

Engineering teams that have experienced production performance regressions from code changes. Automated performance testing catches regressions in CI where they are cheap to fix, rather than in production where they impact customers.

Tools and Technologies

k6, Gatling, Locust, JMeter, Grafana (k6 Cloud or self-hosted), InfluxDB, GitHub Actions, Prometheus, Slack notifications

Implementation Steps

1. Write k6 test scripts that simulate realistic user journeys: browse, search, add to cart, checkout.
2. Define performance thresholds in the k6 script: `http_req_duration p(99) < 500ms`, `error_rate < 1%`.
3. Create a baseline performance profile by running tests against the current production version.
4. In the CI pipeline, add a performance test stage that deploys to a staging environment and runs k6.
5. Compare results against the baseline; fail the pipeline if P99 latency degrades by more than 10%.
6. Stream k6 metrics to Grafana via InfluxDB for real-time visualisation during test runs.
7. Schedule nightly soak tests (4-hour sustained load) to catch memory leaks and resource exhaustion.
8. Store test results in S3 with the commit SHA; build a trend dashboard showing performance over time.

Best Practices

- Test against an environment that mirrors production hardware; test results on undersized hardware are misleading.
- Use realistic data in load tests; synthetic data that hits cache 100% of the time misrepresents production behaviour.
- Test different load shapes: ramp-up, constant, spike, and soak — each reveals different bottlenecks.
- Run performance tests from multiple geographic regions to test global latency, not just data centre latency.
- Profile the application under load (using async profiler or py-spy) to find the hot paths causing slowdowns.

Challenges and Solutions

Challenge

Load tests overwhelm the staging database and corrupt test data.

Solution

Use a separate performance test database with production-like schema and synthetic data; reset it before each test run.

Challenge

Performance test results are highly variable between runs.

Solution

Run 3 consecutive test iterations and compare median results; discard outlier runs caused by infrastructure noise.

Challenge

Load test infrastructure is expensive to run in CI.

Solution

Run lightweight smoke performance tests (1 minute, 10 VUs) on every PR; run full soak tests only on nightly builds.

Project 26: Event-Driven Architecture with Kafka and Kubernetes

Project 26 of 50 • Messaging & Event Streaming

Description

Design and deploy an event-driven microservices architecture using Apache Kafka running on Kubernetes. Services communicate asynchronously through events, enabling decoupled scaling, replay capability, and resilience to temporary service unavailability.

Use Case

E-commerce, fintech, and logistics platforms where multiple downstream systems need to react to the same business event (order placed, payment processed) without tight coupling. Event streaming also enables event sourcing and CQRS patterns for audit-compliant systems.

Tools and Technologies

Apache Kafka, Strimzi (Kafka on Kubernetes), Kafka Connect, Kafka Streams, Schema Registry (Confluent), KSQL, Prometheus JMX Exporter, Grafana, Kafdrop (UI)

Implementation Steps

1. Deploy Kafka on Kubernetes using Strimzi operator; configure 3 broker replicas and 3 ZooKeeper nodes (or KRaft mode).
2. Create topics with appropriate partitions and replication factors; set retention policies per topic.
3. Deploy Confluent Schema Registry; define Avro or Protobuf schemas for all event types.
4. Write producer services that publish events with full schema validation before sending.
5. Write consumer services using consumer groups; implement idempotent processing for at-least-once delivery.
6. Deploy Kafka Connect with JDBC connectors to sync database changes to Kafka topics (CDC).
7. Configure Kafka Streams applications for real-time aggregations and event transformations.
8. Monitor consumer lag with Prometheus and alert when lag exceeds 10,000 messages.

Best Practices

- Treat schema evolution seriously: use compatible schema changes (add optional fields only) and test with the Schema Registry compatibility check.
- Design consumers to be idempotent; duplicate message delivery is expected and must be handled gracefully.
- Set partition counts based on expected parallelism; changing partitions after creation is disruptive.
- Use Kafka transactions for exactly-once semantics when financial accuracy is required.
- Document the event catalogue: every event type, its schema, producer, and consumers — this becomes critical as the system grows.

Challenges and Solutions

Challenge	Solution
-----------	----------

Consumer lag builds up during high load, delaying downstream processing.

Scale consumer instances up to the number of partitions; add partitions and scale consumers proportionally for permanent capacity increases.

Challenge

Schema changes break consumer compatibility.

Solution

Enforce FORWARD_TRANSITIVE compatibility in Schema Registry; perform consumer-first deployments during schema migrations.

Challenge

Debugging distributed event flows across many services is complex.

Solution

Inject correlation IDs and trace headers into Kafka message headers; use distributed tracing to follow event flows.

Project 27: Disaster Recovery and Business Continuity Planning

Project 27 of 50 • Reliability Engineering

Description

Design and implement a disaster recovery (DR) plan with automated failover capabilities. Define and test Recovery Time Objective (RTO) and Recovery Point Objective (RPO) targets. The system automatically detects failures and fails over to the DR region without manual intervention.

Use Case

Any organisation with revenue-critical systems that cannot tolerate extended outages. Banks, healthcare systems, and SaaS companies with SLA commitments need documented and tested DR procedures that can be executed under pressure at 3am.

Tools and Technologies

AWS Route 53 (health checks and failover), AWS RDS Multi-AZ, S3 Cross-Region Replication, Terraform, AWS Backup, CloudFormation StackSets, Runbook automation (Lambda), PagerDuty

Implementation Steps

1. Define RTO and RPO targets per workload tier (Tier 1 = RTO 15min/RPO 1min; Tier 2 = RTO 4hr/RPO 1hr).
2. Provision a DR region with a scaled-down replica of Tier 1 infrastructure using Terraform modules.
3. Configure RDS Multi-AZ for database resilience within a region; set up RDS Read Replica in the DR region.
4. Enable S3 Cross-Region Replication for all application data buckets.
5. Create Route 53 health checks pointing to primary and DR ALBs; configure failover routing policy.
6. Write a failover runbook as a Lambda function: promote RDS replica, update Route 53, scale up DR ASG.
7. Schedule quarterly DR tests: simulate primary region failure and execute the runbook; measure actual RTO vs. target.
8. Produce a post-DR-test report documenting what worked, what failed, and action items.

Best Practices

- A DR plan that is not tested is not a DR plan — test failover at least quarterly; test recovery monthly.
- Automate the failover process: humans under pressure at 3am make mistakes that automation does not.
- Ensure DNS TTLs are set low (30–60 seconds) before a planned failover to minimise propagation delays.
- Keep DR infrastructure costs low by using stopped instances and scaling to full capacity only during failover.
- Train all engineers on the DR runbook, not just the on-call team; cross-functional knowledge reduces risk.

Challenges and Solutions

Challenge	Solution
-----------	----------

RDS promotion to a read replica takes longer than the target RTO.

Pre-warm the DR replica with recent transactions; use Aurora Global Database which promotes in under 1 minute.

Challenge

Application state (sessions, caches) is lost during failover.

Solution

Use a globally distributed cache (DynamoDB Global Tables, ElastiCache Global Datastore) to replicate session data continuously.

Challenge

DR tests disrupt production traffic due to Route 53 changes.

Solution

Use a dedicated DR test subdomain (dr-test.example.com) so DR exercises do not affect real user traffic.

Project 28: Kubernetes Multi-Tenancy with Namespace Isolation

Project 28 of 50 • Container Orchestration

Description

Design and implement a multi-tenant Kubernetes platform where multiple teams or customers share the same cluster with strong isolation boundaries. Each tenant has its own namespace, resource quotas, network policies, and RBAC rules — and cannot access or impact other tenants.

Use Case

Platform teams operating a shared Kubernetes cluster for multiple internal teams or managing a SaaS product where each customer's workloads must be isolated. Multi-tenancy reduces infrastructure cost while maintaining security and fair resource allocation.

Tools and Technologies

Kubernetes (Namespaces, RBAC, Network Policies), OPA Gatekeeper, Kyverno, Cilium (eBPF network policies), HNC (Hierarchical Namespace Controller), Loft (virtual clusters), Crossplane

Implementation Steps

1. Design the namespace structure: one namespace per team per environment (team-a-prod, team-a-staging).
2. Write Terraform that provisions a namespace with ResourceQuota, LimitRange, NetworkPolicy, and RBAC in a single module.
3. Configure NetworkPolicy with a default-deny rule; add explicit allow rules only for required service communication.
4. Deploy OPA Gatekeeper constraints that prevent a tenant from creating ClusterRole or accessing other namespaces.
5. Install HNC to allow tenants to create sub-namespaces for their own sub-teams.
6. Configure PodSecurity admission to enforce baseline or restricted standards per namespace.
7. Set up chargeback using Kubecost namespace grouping so each tenant receives a monthly cost report.
8. Create an onboarding automation: a PR to add a new tenant triggers Terraform to provision everything automatically.

Best Practices

- Never grant cluster-admin to application teams; use namespace-scoped roles and escalate through a reviewed process.
- Set ResourceQuotas conservatively and adjust based on actual usage; over-provisioned quotas lead to noisy-neighbour problems.
- Use Cilium network policies for more expressive east-west traffic control based on DNS names and HTTP methods.
- Audit RBAC permissions quarterly; remove stale service accounts and excessive role bindings.
- Test isolation boundaries: attempt to access another tenant's secrets from within a test pod and verify it fails.

Challenges and Solutions

Challenge

A noisy tenant consuming excessive CPU starves other tenants.

Solution

Set CPU limits on all pods; configure PriorityClasses so critical workloads preempt batch jobs from any tenant.

Challenge

Network policies block legitimate monitoring traffic (Prometheus scraping).

Solution

Create an explicit network policy that allows traffic from the monitoring namespace to all tenant namespaces on metrics ports.

Challenge

Tenants need cluster-scoped resources (CRDs) that require cluster-admin.

Solution

Use virtual clusters (Loft vcluster) for tenants that require cluster-scoped access; they get a full API server with isolated scope.

Project 29: Immutable Infrastructure with Packer and AWS AMIs

Project 29 of 50 • Infrastructure Automation

Description

Build a pipeline that uses Packer to bake application-ready Amazon Machine Images (AMIs). Servers are never modified after launch; when changes are needed, a new AMI is built and instances are replaced. This approach eliminates configuration drift and makes rollbacks trivial.

Use Case

Teams running long-lived EC2 instances that suffer from configuration drift or snowflake servers that cannot be reproduced. Immutable infrastructure ensures that every instance in production is identical to what was tested and eliminates 'but it works on server X' problems.

Tools and Technologies

HashiCorp Packer, Ansible (provisioner), AWS EC2, AWS AMI, Auto Scaling Groups, Launch Templates, GitHub Actions, Terraform, AWS Systems Manager

Implementation Steps

1. Write a Packer HCL template: start from an official Amazon Linux 2 AMI, define an Ansible provisioner.
2. Write Ansible roles that install the application runtime, configure sysctl, harden SSH, and set up the monitoring agent.
3. Create a GitHub Actions workflow: on merge to main, run packer build, capture the new AMI ID.
4. Store the AMI ID as a Terraform variable; create a PR to the infrastructure repo updating the launch template.
5. On infrastructure PR merge, Terraform updates the Auto Scaling Group launch template with the new AMI.
6. Trigger an ASG instance refresh: AWS replaces instances in batches without downtime.
7. Test the new AMI in a staging ASG first; compare CloudWatch metrics before promoting to production.
8. Set up an AMI lifecycle policy: retain the last 5 AMIs and deregister older ones to control costs.

Best Practices

- Bake all application dependencies and configurations into the AMI; never rely on userdata scripts for critical setup.
- Run CIS benchmark hardening and security scanning (Inspector) against the AMI in the build pipeline.
- Tag AMIs with the Git commit SHA, build date, and base AMI ID for full traceability.
- Test the AMI boot time and health check pass time; a slow-to-start instance extends the deployment window.
- Keep AMI bake times under 15 minutes by caching large downloads and using pre-installed base layers.

Challenges and Solutions

Challenge	Solution
-----------	----------

AMI builds are slow (20–30 minutes) making the pipeline feel sluggish.

Use a base AMI with common software pre-installed; only run application-specific steps in the final Packer build.

Challenge

Configuration changes in Ansible break the AMI build mid-pipeline.

Solution

Test Ansible roles locally using Molecule before committing; run ansible-lint in the pipeline as a pre-check.

Challenge

Rolling out a new AMI across a large ASG takes too long.

Solution

Tune the instance refresh MinHealthyPercentage and InstanceWarmup for your scaling group size and application startup time.

Project 30: Kubernetes Cost Optimisation with Vertical and Horizontal Autoscaling

Project 30 of 50 • FinOps

Description

Implement intelligent autoscaling in Kubernetes using Vertical Pod Autoscaler (VPA) for right-sizing and Horizontal Pod Autoscaler (HPA) for scaling out. Combine with Cluster Autoscaler and Spot Instances to reduce Kubernetes running costs by 40–60% without impacting performance.

Use Case

Platform teams running over-provisioned Kubernetes clusters where most pods use less than 20% of their requested CPU and memory. Autoscaling eliminates the need to manually tune resource requests and reduces over-provisioning waste.

Tools and Technologies

Kubernetes HPA, VPA, Cluster Autoscaler, KEDA (event-driven autoscaling), AWS Karpenter, Spot Instance diversification, Kubecost, Prometheus Adapter (custom metrics HPA)

Implementation Steps

1. Deploy VPA in recommendation mode first; do not auto-apply changes until you understand the recommendations.
2. Review VPA recommendations for each deployment; update resource requests based on observed usage.
3. Configure HPA with CPU target at 70% utilisation; configure scale-down stabilisation window to avoid flapping.
4. Install KEDA for event-driven scaling: scale Kafka consumers based on consumer lag, HTTP scalers based on queue depth.
5. Deploy Karpenter (or Cluster Autoscaler) to automatically provision and deprovision nodes.
6. Configure Karpenter node provisioner to use a mix of On-Demand and Spot Instances (70/30 split).
7. Set up Spot Instance interruption handling: pre-stop lifecycle hooks drain in-flight requests.
8. Monitor scaling events in Grafana; track cost/performance ratio over 30 days.

Best Practices

- Never run VPA and HPA on CPU/memory for the same deployment simultaneously; they conflict. Use KEDA + VPA instead.
- Set Spot Instances on stateless, fault-tolerant workloads only; keep On-Demand for stateful services.
- Use Karpenter's consolidation feature to bin-pack pods onto fewer nodes and terminate underutilised ones.
- Review resource request recommendations quarterly and update them; VPA in recommendation mode provides the data.
- Set namespace-level LimitRange defaults so new workloads without explicit limits are still bounded.

Challenges and Solutions

Challenge HPA scales down too aggressively during brief traffic lulls, causing cold start latency.	Solution Set <code>scaleDownStabilizationWindowSeconds</code> to 300 seconds; keep minimum replicas at 2 to preserve warm capacity.
Challenge Spot Instances are interrupted during peak traffic causing availability issues.	Solution Diversify across 5+ instance types and 3 availability zones; Karpenter's spot interruption handling ensures graceful drainage.
Challenge VPA recommendations conflict with JVM heap settings.	Solution Exclude memory from VPA management for JVM workloads; only use VPA for CPU recommendations and manage JVM heap manually.

Project 31: API Gateway and Rate Limiting with Kong

Project 31 of 50 • Networking & Service Mesh

Description

Deploy Kong API Gateway as the entry point for all external API traffic. Configure authentication, rate limiting, request transformation, and routing rules centrally rather than in each microservice. The gateway enforces consistent cross-cutting concerns so individual services can focus on business logic.

Use Case

Organisations exposing APIs to third-party partners or building internal API platforms. A centralised gateway prevents rate limit abuse, adds authentication without modifying every service, and provides a single point for API analytics and monetisation.

Tools and Technologies

Kong Gateway, Kong Ingress Controller, PostgreSQL (Kong database), Deck (Kong config as code), Prometheus Kong plugin, Keycloak (OIDC), GitHub Actions, Grafana

Implementation Steps

1. Deploy Kong using the Kong Ingress Controller on Kubernetes with a PostgreSQL backend.
2. Configure routes for each backend service in Kong, pointing to Kubernetes Services.
3. Enable the Rate Limiting Advanced plugin: 100 req/min per consumer, 1000 req/min per IP.
4. Add the JWT plugin with Keycloak as the OIDC provider; Kong validates tokens before forwarding requests.
5. Configure the Prometheus plugin to expose Kong metrics (latency, status codes, upstream health).
6. Use Deck (deckK) to manage Kong configuration as YAML files in Git; apply changes via CI pipeline.
7. Set up Kong's health checks: active probes on upstream services; remove unhealthy upstreams from rotation.
8. Build a Grafana dashboard with Kong metrics: request rate, error rate, P99 latency per route.

Best Practices

- Manage all Kong configuration through Deck in Git; never use the Admin API directly in production.
- Apply rate limits by consumer (API key holder) and by IP separately to handle both bot abuse and individual user abuse.
- Use Kong's request and response transformation plugins to version APIs gracefully without modifying upstream services.
- Enable Kong's correlation ID plugin to inject a unique request ID into every request header for tracing.
- Test Kong configuration changes in a staging gateway before applying to production using the Deck diff command.

Challenges and Solutions

Challenge	Solution
-----------	----------

Kong configuration becomes complex with hundreds of routes and plugins.

Organise configuration into service-specific deck files; use a Kong control plane / data plane separation for large deployments.

Challenge

Rate limit plugin causes false positives for legitimate burst traffic.

Solution

Use the sliding window algorithm; increase burst limits for authenticated consumers with known usage patterns.

Challenge

PostgreSQL becomes a single point of failure for Kong.

Solution

Run PostgreSQL with synchronous replication or use a managed RDS PostgreSQL Multi-AZ instance.

Project 32: Continuous Security Scanning with Falco Runtime Security

Project 32 of 50 • Security & DevSecOps

Description

Deploy Falco as a runtime security monitor across the Kubernetes cluster. Falco watches system calls in real time and alerts on suspicious behaviour — unexpected privilege escalation, shell spawned in a container, file system writes to unexpected paths — enabling detection of and response to active attacks.

Use Case

Security teams that need runtime visibility into containerised workloads. Traditional scanning finds vulnerabilities in images before deployment, but Falco catches malicious activity after deployment — supply chain attacks, container escape attempts, and insider threats.

Tools and Technologies

Falco, Falco Sidekick (alert routing), Kubernetes, Prometheus (Falco metrics), Slack, PagerDuty, AWS Security Hub, Elasticsearch (Falco audit logs), Helm

Implementation Steps

1. Install Falco using Helm as a DaemonSet with the kernel module or eBPF driver for system call tracing.
2. Deploy Falco Sidekick to route alerts to Slack, PagerDuty, and AWS Security Hub simultaneously.
3. Review the default Falco ruleset; disable overly noisy rules that do not apply to your environment.
4. Write custom rules for your application context: alert on writes to /etc/, reads of /proc/<pid>/maps outside expected tools.
5. Integrate Falco alerts with the incident management workflow via PagerDuty.
6. Stream Falco audit events to Elasticsearch for long-term retention and forensic analysis.
7. Create a Grafana dashboard showing Falco event rate by rule priority and namespace.
8. Conduct a red team exercise: attempt a container escape and verify Falco fires the correct alert.

Best Practices

- Tune rules aggressively in the first 30 days to eliminate false positives; noisy alerts are ignored.
- Treat a Falco CRITICAL alert as a potential security incident; define and practice the response procedure.
- Use Falco in combination with network policies and image signing; runtime security is one layer of defence in depth.
- Enrich Falco alerts with Kubernetes context (pod name, deployment, namespace) for faster triage.
- Review and update custom rules quarterly as application behaviour evolves.

Challenges and Solutions

Challenge	Solution
-----------	----------

Falco kernel module compatibility breaks after node OS upgrades.

Use the eBPF driver (falco-ebpf) which is more portable; set up automated driver testing in a staging node pool.

Challenge

Legitimate application behaviour triggers Falco false positives.

Solution

Use exception lists in Falco rules; tune by container image and process name to scope rules precisely.

Challenge

High event volume overwhelms downstream SIEM.

Solution

Set minimum rule priority to WARNING in production; send DEBUG events only to Elasticsearch for offline analysis.

Project 33: GitLab CI/CD Self-Hosted Pipeline

Project 33 of 50 • Continuous Integration & Delivery

Description

Set up a self-hosted GitLab instance with integrated CI/CD, container registry, security scanning, and merge request workflows. The platform provides a single tool for source control, CI/CD, and security, reducing the number of integrations teams need to manage.

Use Case

Organisations that cannot use cloud-hosted CI/CD due to data residency requirements, airgapped environments, or a desire for tighter integration between source control and pipelines. Self-hosted GitLab provides enterprise features at lower per-seat cost than comparable cloud alternatives.

Tools and Technologies

GitLab (Omnibus or Helm chart), GitLab Runner (Docker executor), PostgreSQL, Redis, MinIO (object storage), GitLab Container Registry, SAST/DAST (GitLab built-in), Prometheus, Grafana

Implementation Steps

1. Install GitLab using the Omnibus package on Ubuntu or the official Helm chart on Kubernetes.
2. Configure external PostgreSQL and Redis for HA; use MinIO or S3 for object storage.
3. Deploy GitLab Runner instances with Docker executor; register runners to the GitLab instance.
4. Write a .gitlab-ci.yml with stages: build, test, security-scan, build-image, deploy.
5. Enable built-in SAST, Secret Detection, and Dependency Scanning templates in the pipeline.
6. Configure merge request pipelines and branch protection: require passing pipeline and code review.
7. Integrate GitLab with Jira for issue tracking and Slack for pipeline notifications.
8. Monitor GitLab performance with the built-in Prometheus metrics and configure Grafana dashboards.

Best Practices

- Enable GitLab's Auto DevOps as a starting point; customise the generated pipeline rather than writing from scratch.
- Use include: templates: to pull in security scanning templates centrally; avoid duplicating scan config in every repo.
- Configure GitLab Ultimate's security dashboard to aggregate findings across all projects for the security team.
- Run GitLab on dedicated nodes separate from application workloads to prevent resource contention.
- Back up GitLab configuration, database, and object storage daily; test restore monthly.

Challenges and Solutions

Challenge

GitLab storage grows rapidly with large repositories and artifacts.

Solution

Enable GitLab artifact expiry policies; configure object lifecycle rules in MinIO/S3 to delete expired artifacts automatically.

Challenge

Runner queues back up during peak build times.

Solution

Configure runner autoscaling using the GitLab Runner docker-machine executor on AWS Spot; scale down during off-hours.

Challenge

GitLab upgrades require careful planning to avoid breaking pipelines.

Solution

Follow the GitLab upgrade path documentation; test upgrades in a staging instance before production. Never skip major versions.

Project 34: Zero-Trust Network Security Implementation

Project 34 of 50 • Security & DevSecOps

Description

Implement a zero-trust security model where no network traffic is trusted by default — every request is authenticated, authorised, and encrypted regardless of origin. Combine service mesh mTLS, identity-based access, and micro-segmentation to eliminate the implicit trust of traditional perimeter-based security.

Use Case

Enterprises responding to the reality that perimeter security is insufficient — internal threats, stolen credentials, and lateral movement attacks prove that being inside the network does not mean being trusted. Zero trust is now required for FedRAMP, NIST 800-207, and many enterprise security frameworks.

Tools and Technologies

Istio (mTLS), SPIFFE/SPIRE (workload identity), HashiCorp Vault, Cloudflare Access (Zero Trust Network Access), AWS VPC Lattice, OPA Gatekeeper, CyberArk PAM

Implementation Steps

1. Deploy SPIRE as the identity authority; configure SPIFFE IDs for every workload based on Kubernetes service account.
2. Enable Istio in STRICT mTLS mode; all in-mesh traffic is mutually authenticated using SPIFFE certificates.
3. Replace long-lived service credentials with short-lived tokens issued by Vault using workload identity.
4. Configure Cloudflare Access for all internal HTTP services; employees authenticate with SSO + device posture check.
5. Write OPA policies that authorise service-to-service calls based on SPIFFE identity, not IP address.
6. Deploy AWS VPC Lattice for zero-trust service-to-service communication outside Kubernetes.
7. Audit all access: log every service call decision to the SIEM for compliance and threat hunting.
8. Conduct a purple team exercise: simulate lateral movement and verify that zero-trust controls contain it.

Best Practices

- Start with the most sensitive data paths (payment, PII) first; expand to all services incrementally.
- Automate certificate rotation; manually managed certificates expire unexpectedly at the worst times.
- Use identity-based policies, not IP-based; IPs change with cloud elasticity but identity is stable.
- Instrument every policy decision point; the ability to audit who accessed what is as important as prevention.
- Run tabletop exercises: simulate a compromised service account and walk through the blast radius under zero trust.

Challenges and Solutions

Challenge	Solution
-----------	----------

Legacy applications cannot participate in SPIFFE/mTLS without modification.

Place a SPIRE-aware sidecar (Envoy) in front of legacy services; the proxy handles identity and mTLS on their behalf.

Challenge

SPIFFE certificate rotation causes brief TLS handshake failures.

Solution

Configure graceful rotation: issue new certificates 5 minutes before expiry and keep old certs valid for an overlap period.

Challenge

Zero trust adds latency to every inter-service call.

Solution

Benchmark the overhead (typically 1–2ms for mTLS handshake, then negligible); use session resumption to amortise cost.

Project 35: Cloud-Native Backup and Data Protection

Project 35 of 50 • Reliability Engineering

Description

Implement a comprehensive backup strategy for cloud-native workloads including databases, Kubernetes persistent volumes, application state, and configuration. Automate backup scheduling, retention, encryption, and restoration testing so the team has confidence in their data protection posture.

Use Case

Teams that have experienced data loss or need to demonstrate to auditors that they have a tested backup and recovery capability. Compliance frameworks (HIPAA, SOC 2, PCI DSS) require documented, regularly-tested backup procedures.

Tools and Technologies

Velero (Kubernetes backup), AWS Backup, RDS automated backups, AWS DLM (EBS snapshots), Restic, S3 (backup storage), KMS (backup encryption), GitHub Actions (backup verification)

Implementation Steps

1. Deploy Velero in the Kubernetes cluster; configure S3 as the backup storage location with KMS encryption.
2. Create Velero backup schedules: hourly incremental with 24-hour retention, daily with 30-day retention.
3. Configure AWS Backup plans covering RDS, DynamoDB, EFS, and EBS volumes with lifecycle rules.
4. Set up cross-region backup replication for Tier 1 data: primary backups replicate to a DR region.
5. Write a backup verification Lambda that runs nightly: restores a backup to an isolated environment and validates data integrity.
6. Test application-consistent backups: use pre/post backup hooks in Velero to quiesce application writes.
7. Document the restore procedure for each data source; measure and record actual restore times.
8. Build a backup compliance dashboard showing coverage, last backup time, and restore test results.

Best Practices

- The 3-2-1 rule: 3 copies, 2 different media, 1 offsite. For cloud: 3 snapshots, 2 AWS regions, 1 separate account.
- Encrypt all backups with customer-managed KMS keys; store key material separately from the backup data.
- Test restores monthly, not annually; many teams discover their backups are broken only when they need them.
- Include application configuration and Kubernetes manifests in backups, not just data volumes.
- Automate restore testing as part of the CI pipeline; a backup that cannot be restored is worthless.

Challenges and Solutions

Challenge	Solution
-----------	----------

Velero backups are slow for large persistent volumes.

Use Velero's CSI plugin for volume snapshots instead of the legacy restic method; snapshots are near-instantaneous.

Challenge

Cross-region backup costs are high.

Solution

Apply lifecycle policies: full backups in the primary region, compressed archives in S3 Glacier in the DR region.

Challenge

Restoring a Kubernetes namespace with dependencies on cluster-scoped resources fails.

Solution

Document namespace dependencies; use Velero include-cluster-resources flag and restore in the correct order.

Project 36: Developer Environment Automation with Dev Containers

Project 36 of 50 • Developer Experience

Description

Standardise development environments using VS Code Dev Containers and Docker Compose, ensuring every developer works in an identical, reproducible environment with all required tools pre-installed. Onboarding time drops from days to under 30 minutes.

Use Case

Engineering teams tired of 'works on my machine' problems and spending days setting up new developer environments. Dev containers ensure the environment matches CI and production, reducing a class of bugs caused by local environment differences.

Tools and Technologies

VS Code Dev Containers, Docker, Docker Compose, devcontainer.json, GitHub Codespaces, dotfiles, pre-commit hooks, Makefile, Homebrew (for non-container tools)

Implementation Steps

1. Create a `.devcontainer/devcontainer.json` in each repository defining the base image, extensions, and lifecycle scripts.
2. Write a `docker-compose.yml` within `.devcontainer/` including the dev container and all dependent services (database, cache, mock services).
3. Configure `postCreateCommand` in `devcontainer.json` to run setup scripts: install dependencies, initialise database, run migrations.
4. Pre-install all required VS Code extensions in the `devcontainer.json` extensions list.
5. Create a `Makefile` with developer commands: `make dev` (start all services), `make test`, `make migrate`.
6. Configure pre-commit hooks that run linting and formatting automatically before each commit.
7. Test the dev container on a fresh machine by running it without any pre-existing setup; measure time to first successful test run.
8. Publish the dev container image to ECR; developers pull it rather than building it every time.

Best Practices

- Include the dev container setup in the PR review checklist: if your change requires a tool, update the `devcontainer.json`.
- Keep dev container images updated with weekly base image rebuilds to maintain parity with CI.
- Use GitHub Codespaces as a cloud-hosted fallback for developers with constrained local machines.
- Mount the developer's `.gitconfig` and SSH keys into the container so Git workflows feel native.
- Separate the build tools layer from the application layer in the `Dockerfile` to enable faster rebuilds.

Challenges and Solutions

Challenge

Dev containers are slow to start on machines with limited resources.

Solution

Pre-build and publish the dev container image; running a pre-built image is 10x faster than building locally.

Challenge

File system performance inside containers is poor on macOS.

Solution

Use Docker Desktop's VirtioFS mount strategy; avoid mounting the entire project — use named volumes for `node_modules`.

Challenge

Developers using different IDEs cannot use the VS Code devcontainer format.

Solution

Provide a base `docker-compose.yml` that works with any editor alongside the VS Code-specific `devcontainer.json` configuration.

Project 37: SRE Practices: SLIs, SLOs, and Error Budgets

Project 37 of 50 • Reliability Engineering

Description

Implement Site Reliability Engineering (SRE) practices by defining Service Level Indicators (SLIs), Service Level Objectives (SLOs), and error budgets for production services. Error budgets create a shared framework between product and engineering that objectively determines when to prioritise reliability work over new features.

Use Case

Engineering organisations that want a data-driven approach to reliability. Without SLOs, reliability discussions are subjective and often dominated by the loudest voice. SLOs and error budgets replace opinion with measurement, aligning incentives between product and reliability work.

Tools and Technologies

Prometheus, Grafana (error budget dashboards), Pyrra or Sloth (SLO generators), OpenSLO spec, PagerDuty, Slack, Confluence (SLO documentation)

Implementation Steps

1. Define SLIs for each critical user journey: availability (ratio of successful requests), latency (proportion under 500ms), correctness.
2. Set SLO targets based on user expectations and current reliability data: e.g., 99.9% availability over 30 days.
3. Calculate the error budget: $100\% - 99.9\% = 0.1\%$ of 30 days = 43.2 minutes of allowed downtime per month.
4. Deploy Pyrra to generate Prometheus recording rules and Grafana dashboards from SLO YAML definitions.
5. Create an error budget burn rate dashboard: alert when burn rate exceeds 2x (suggesting budget will exhaust early).
6. Establish the error budget policy: if budget < 50%, freeze non-essential feature releases; if < 0%, focus entirely on reliability.
7. Review SLO attainment in weekly engineering reviews; share the dashboard with the product team.
8. Run an SLO postmortem whenever an SLO is breached; identify root causes and reliability improvements.

Best Practices

- Set SLOs slightly below what you can actually achieve; you need headroom to deploy and maintain the service.
- Define SLIs from the user perspective: measure at the load balancer, not deep in the stack where latency is hidden.
- Make error budgets visible to product managers; they will naturally deprioritise risky features when budget is low.
- Review and adjust SLOs annually as the product and user expectations evolve.
- Start with 1–2 SLOs per service; avoid the temptation to measure everything — quality over quantity.

Challenges and Solutions

Challenge

Disagreement on what constitutes a 'good' request for the SLI.

Solution

Define explicitly in writing: exclude health check traffic, exclude requests that return 4xx due to client error, include timeout errors.

Challenge

Error budget policy is ignored by product managers focused on deadlines.

Solution

Get executive sponsorship; frame error budget policy as a business risk management tool, not a technical constraint.

Challenge

SLO dashboards are too complex for non-engineers to understand.

Solution

Build a simplified exec dashboard: a single traffic light per service showing SLO status for the current period.

Project 38: Automated Kubernetes Cluster Upgrades

Project 38 of 50 • Container Orchestration

Description

Build a fully automated process for upgrading Kubernetes clusters with zero downtime. Control plane upgrades, node group rolling upgrades, and add-on updates are orchestrated in sequence with automated testing and rollback capability at every step.

Use Case

Platform teams managing multiple EKS clusters who need to stay within the Kubernetes support window (N-2 minor versions) without scheduling weekend maintenance windows. Automated upgrades reduce upgrade anxiety and keep clusters secure and supported.

Tools and Technologies

AWS EKS, eksctl, AWS CLI, kubectl, Helm, GitHub Actions, Pytest (cluster validation), Slack notifications, Terraform (node group management)

Implementation Steps

1. Create a GitHub Actions workflow scheduled monthly triggered by a version check against the EKS release calendar.
2. Stage 1: Upgrade the EKS control plane using the AWS API; monitor until the upgrade completes.
3. Stage 2: Update EKS managed add-ons (CoreDNS, kube-proxy, vpc-cni) to the versions required by the new control plane.
4. Stage 3: Update Helm charts for cluster-level components (Karpenter, ALB Controller, cert-manager) to compatible versions.
5. Stage 4: Upgrade node groups using `eksctl upgrade nodegroup --force-upgrade` with a rolling update strategy.
6. Run a validation suite after each stage: check node readiness, run `kubectl get pods --all-namespaces`, execute a synthetic request.
7. If validation fails at any stage, trigger an alert and pause the workflow for manual review.
8. Test the upgrade process monthly in a staging cluster before applying to production.

Best Practices

- Never skip Kubernetes minor versions; upgrade N-1 at a time following the official upgrade path.
- Check add-on and Helm chart compatibility before upgrading; most outages during upgrades are from incompatible add-ons.
- Upgrade staging clusters 2 weeks before production; run full regression test suites after staging upgrades.
- Use Pod Disruption Budgets on all critical workloads so rolling node upgrades respect availability requirements.
- Document the upgrade history in a CHANGELOG: date, version before, version after, issues encountered.

Challenges and Solutions

Challenge

A Helm chart is incompatible with the new Kubernetes version and fails after the node upgrade.

Solution

Run Pluto and Kubent pre-upgrade to detect deprecated APIs used by running resources and Helm chart manifests.

Challenge

Node group upgrade drains nodes faster than applications can gracefully terminate.

Solution

Configure preStopHook and terminationGracePeriodSeconds to ensure graceful shutdown; set PodDisruptionBudgets appropriately.

Challenge

Manual approval is required for production but the team is in different time zones.

Solution

Build a Slack approval workflow: the pipeline posts a message with Approve/Reject buttons connected to GitHub Actions manual approval.

Project 39: Multi-Region Active-Active Application Deployment

Project 39 of 50 • Reliability Engineering

Description

Deploy an application in active-active mode across two AWS regions. Both regions serve production traffic simultaneously; traffic is routed by latency-based DNS. If one region fails, the other absorbs all traffic automatically with no manual intervention required.

Use Case

Global applications where user latency matters and single-region outages cannot be tolerated. Financial trading platforms, global SaaS products, and healthcare systems processing time-sensitive data need multi-region active-active for both performance and resilience.

Tools and Technologies

AWS Route 53 (latency routing), AWS Global Accelerator, DynamoDB Global Tables, Aurora Global Database, CloudFront, Terraform, AWS CloudWatch, Datadog

Implementation Steps

1. Design the data architecture: identify data that must be globally consistent versus data that can be eventually consistent.
2. Deploy DynamoDB Global Tables or Aurora Global Database for shared application state; configure read/write endpoints per region.
3. Provision identical application stacks in two regions using Terraform with region as a variable.
4. Configure Route 53 latency-based routing with health checks; Route 53 removes an unhealthy region automatically.
5. Enable AWS Global Accelerator for consistent, low-latency entry points regardless of DNS caching.
6. Configure CloudFront with multiple origins (one per region) for static assets and cacheable API responses.
7. Instrument Region awareness in the application: log which region served each request for capacity analysis.
8. Run quarterly active-active failover tests: simulate a region failure and verify automatic traffic shift.

Best Practices

- Design for conflict resolution: concurrent writes to the same item in two regions require application-level conflict handling.
- Test replication lag under load; globally consistent databases add write latency — profile this under production-like conditions.
- Use regional health checks with synthetic monitoring, not just infrastructure checks, to detect application-level failures.
- Keep region configurations identical using infrastructure-as-code; divergence between regions causes split-brain behaviour.
- Monitor cross-region replication lag as a key metric; alert if lag exceeds defined thresholds.

Challenges and Solutions

Challenge

DynamoDB Global Tables replication conflicts when two users write to the same key simultaneously.

Solution

Use optimistic locking with a version attribute; the conflict resolution strategy (last-write-wins vs. custom) must be a conscious design decision.

Challenge

Cost doubles because every resource runs in two regions.

Solution

Right-size the second region based on actual failover capacity needs, not a full mirror; use 50% capacity with autoscaling headroom.

Challenge

Debugging an incident is complex when traffic is split across regions.

Solution

Centralise logs and traces from both regions into a single observability platform; tag every telemetry item with its source region.

Project 40: Automated Terraform Drift Detection and Remediation

Project 40 of 50 • Infrastructure Automation

Description

Build a continuous drift detection system that compares live cloud infrastructure against the Terraform state, reports drift, and can automatically remediate it. Teams get early warning when manual changes or external processes modify managed infrastructure.

Use Case

Operations teams fighting configuration drift in large cloud environments. Manual changes made during incidents, configuration pushed by compliance tools, or changes made through the AWS console silently diverge from Terraform state, making the next terraform apply potentially destructive.

Tools and Technologies

Terraform, Terragrunt, Atlantis, AWS Config, GitHub Actions, Slack, Jira (drift tickets), driftctl (drift analysis tool), AWS Systems Manager

Implementation Steps

1. Schedule a GitHub Actions workflow to run terraform plan --detailed-exitcode on a cron schedule (every 4 hours).
2. Parse the plan output using a Python script; categorise changes as additions, modifications, or deletions.
3. Post a Slack message with a summary of detected drift; include the resource type, attribute, and current vs. expected value.
4. Create a Jira ticket for each drift incident; assign to the team owning the affected resource based on Terraform tags.
5. For approved auto-remediation scenarios (tag drift, minor configuration drift), trigger automatic terraform apply.
6. For significant drift (security group changes, IAM changes), require human review and approval before applying.
7. Build a drift dashboard in Grafana showing drift incidents over time, time-to-remediation, and top drifting resource types.
8. Run driftctl weekly for a deeper analysis that catches resources not managed by Terraform at all.

Best Practices

- Treat drift alerts with urgency; unreviewed drift accumulates and eventually causes a catastrophic apply.
- Root-cause every drift incident: if the same type of resource drifts repeatedly, add a Config rule to prevent manual changes.
- Use AWS Config rules to prevent out-of-band changes to critical resources; make Terraform the only path for sensitive infrastructure.
- Separate drift detection (report only) from drift remediation (apply); remediation requires explicit approval.
- Include drift metrics in the engineering quality dashboard alongside deployment frequency and test coverage.

Challenges and Solutions

Challenge

Terraform plan generates false positives for resources with non-deterministic attributes.

Solution

Use ignore_changes lifecycle arguments for attributes like last-modified timestamps that AWS updates automatically.

Challenge

Drift remediation apply destroys manually created resources that were added for a valid reason.

Solution

Import the resource into Terraform state before running apply; never let drift remediation destroy untracked resources.

Challenge

Large environments generate too many drift alerts to act on.

Solution

Prioritise drift by resource type: IAM, security groups, and routing tables are critical; tag drift is cosmetic. Filter alert routing accordingly.

Project 41: DataOps Pipeline for Analytics Infrastructure

Project 41 of 50 • DataOps

Description

Build a modern DataOps pipeline that treats data infrastructure and data transformation pipelines with the same rigour as application code — version control, automated testing, CI/CD, and observability. The result is reliable, trustworthy data that analysts and data scientists can use with confidence.

Use Case

Data engineering teams struggling with broken dashboards, unreliable data pipelines, and no visibility into data quality. DataOps practices reduce data incidents, accelerate time to insight, and build trust between data teams and business stakeholders.

Tools and Technologies

dbt (data transformation), Apache Airflow, Great Expectations (data quality), GitHub Actions, Snowflake / BigQuery / Redshift, Monte Carlo (data observability), Terraform (data infrastructure), Metabase / Looker

Implementation Steps

1. Adopt dbt for all SQL transformations; structure models in layers: staging → intermediate → marts.
2. Write dbt tests for every model: not-null, unique, accepted values, and referential integrity.
3. Deploy Apache Airflow on Kubernetes using the official Helm chart; define DAGs in Python stored in Git.
4. Create a CI pipeline: on every PR, run dbt compile, dbt test on a development schema, and check for breaking changes.
5. Integrate Great Expectations to validate data quality at pipeline boundaries; fail the pipeline on schema or statistical anomalies.
6. Use dbt docs generate to publish a data catalogue with lineage graphs; share with analysts.
7. Configure Monte Carlo (or elementary-data) for production data observability: alert on volume drops, null rate changes, schema changes.
8. Build a DataOps health dashboard: pipeline success rate, data freshness, and test pass rate per data product.

Best Practices

- Treat data transformation SQL as first-class code: code review, unit tests, and version control are non-negotiable.
- Build source freshness tests into Airflow: fail the DAG if upstream source data has not been updated within expected windows.
- Separate raw, staging, and marts layers in your data warehouse; never query raw tables from dashboards.
- Document every model, column, and business metric in dbt's YAML documentation; this becomes the single source of truth.
- Practice data incident management: define SLOs for data freshness and establish a clear process for data incident response.

Challenges and Solutions

Challenge

Long dbt model run times slow down the CI pipeline.

Solution

Use dbt's `--select` flag to run only modified models and their dependents in CI; run the full model graph only in production.

Challenge

Breaking schema changes in source systems silently break downstream models.

Solution

Configure source freshness and schema change alerts in Monte Carlo; add contract tests that fail immediately on upstream schema changes.

Challenge

Data scientists bypass the governed pipeline and query raw tables directly.

Solution

Apply row-level security and column masking to raw schemas; expose governed mart tables to analysts and data scientists instead.

Project 42: Microservices Communication with gRPC and Protobuf

Project 42 of 50 • Networking & Service Mesh

Description

Replace REST JSON APIs between internal microservices with gRPC and Protocol Buffers. gRPC provides strongly typed contracts, efficient binary serialisation, bidirectional streaming, and built-in code generation — resulting in 5-10x lower payload sizes and faster service communication.

Use Case

Engineering teams building high-performance internal service communication where JSON overhead is significant. gRPC is ideal for polyglot microservices environments where teams use different programming languages but need to share a common API contract.

Tools and Technologies

gRPC, Protocol Buffers (protobuf), buf (protobuf toolchain), GitHub Actions, Buf Schema Registry, gRPC-Gateway (REST transcoding), Envoy (gRPC load balancing), grpc_health_probe

Implementation Steps

1. Centralise all protobuf definitions in a dedicated protos/ repository using the buf tool for linting and building.
2. Define service and message types in .proto files; use buf lint to enforce style conventions.
3. Configure buf generate to produce language-specific client and server stubs for Go, Python, Java, and Node.js.
4. Publish generated code to language-specific package registries (npm, PyPI, Maven) via CI pipeline.
5. Set up breaking change detection with buf breaking in PR pipeline; block PRs that introduce incompatible changes.
6. Deploy gRPC-Gateway alongside each service to transcode HTTP/JSON requests to gRPC for external clients.
7. Configure health checking using the gRPC health protocol; integrate with Kubernetes readiness probes.
8. Monitor gRPC metrics (status codes, latency) using the Envoy metrics exposed to Prometheus.

Best Practices

- Treat .proto files as a shared API contract — changes must be backward compatible and follow a defined deprecation process.
- Use deadlines in every gRPC call; without them, clients can hang indefinitely waiting for slow servers.
- Version services using protobuf package names (v1, v2); never break existing clients silently.
- Enable gRPC reflection in development environments for tooling support; disable in production for security.
- Write contract tests that verify client and server implementations against the same proto definition.

Challenges and Solutions

Challenge gRPC services are difficult to test with standard HTTP tools like curl.	Solution Use grpcurl for command-line testing; use BloomRPC or Postman's gRPC support for GUI-based testing.
Challenge Proto schema changes break existing clients in polyglot environments.	Solution Use buf schema registry to track compatibility; enforce WIRE_JSON compatibility mode and require explicit deprecation of old fields.
Challenge Load balancers (AWS ALB) do not natively support gRPC connection-level load balancing.	Solution Use gRPC's built-in client-side load balancing with headless Kubernetes Service; or use Envoy as a gRPC-aware proxy.

Project 43: Cloud Security Posture Management (CSPM) Implementation

Project 43 of 50 • Security & DevSecOps

Description

Implement a Cloud Security Posture Management solution that continuously audits AWS, Azure, and GCP resources against security best practices and compliance frameworks (CIS Benchmarks, NIST, PCI DSS). Findings are automatically triaged, prioritised, and tracked through to remediation.

Use Case

Security teams responsible for cloud compliance who need visibility across thousands of cloud resources without manually reviewing each one. CSPM replaces point-in-time audit checks with continuous monitoring, dramatically reducing the time to detect and fix misconfigurations.

Tools and Technologies

AWS Security Hub, AWS Config, Prowler, Wiz, Prisma Cloud, Checkov, Terraform Compliance, AWS GuardDuty, Jira (finding tracking), Slack

Implementation Steps

1. Enable AWS Security Hub and connect AWS Config, GuardDuty, and Inspector as finding sources.
2. Onboard all AWS accounts to Security Hub using a delegated administrator account via AWS Organisations.
3. Run Prowler against the AWS environment to generate a baseline CIS Benchmark compliance report.
4. Integrate Prowler into a weekly scheduled GitHub Actions job; publish results to Security Hub.
5. Configure Security Hub to route CRITICAL findings to PagerDuty and HIGH findings to a Jira project.
6. Add Checkov to the CI/CD pipeline to catch misconfigurations before infrastructure is deployed.
7. Enable AWS Config conformance packs for PCI DSS or CIS Level 2; AWS evaluates all resources automatically.
8. Build a compliance dashboard in Security Hub showing findings by severity, account, and compliance standard.

Best Practices

- Suppress findings that are accepted risks with documented justification and expiry dates; unsuppressed findings reflect unreviewed risk.
- Integrate CSPM findings into the development workflow: show findings in the IDE via IDE plugins so developers see misconfigurations early.
- Use CSPM findings to drive infrastructure improvements, not just compliance reporting; track fix rates as a team KPI.
- Correlate CSPM findings with actual exploitability data; a finding on an unexposed resource has different urgency than one on a public endpoint.
- Run quarterly CSPM programme reviews: assess finding volumes, fix SLAs, and tune the tool configuration.

Challenges and Solutions

Challenge

CSPM tools generate thousands of findings, creating overwhelming noise.

Solution

Prioritise by severity AND exposure: critical findings on internet-facing resources first. Suppress known-accepted risks immediately.

Challenge

Developers see CSPM as a compliance burden rather than helpful feedback.

Solution

Shift CSPM left: show findings in PR checks before infrastructure deploys, not after. Frame findings as security improvements.

Challenge

Custom cloud architectures trigger false positives in generic CSPM rules.

Solution

Write custom AWS Config rules for organisation-specific architectures; suppress false positives with documented evidence.

Project 44: AI/ML Model Deployment with MLflow and Kubernetes

Project 44 of 50 • MLOps

Description

Build an MLOps pipeline that tracks experiments, versions models, automates testing, and deploys machine learning models to production Kubernetes as REST APIs. Models are treated like software — versioned, tested, deployed with CI/CD, and monitored in production.

Use Case

Data science teams that build high-quality models but struggle to get them reliably into production. MLOps bridges the gap between data science notebooks and production systems, reducing time from model training to production deployment from months to days.

Tools and Technologies

MLflow (experiment tracking and model registry), Kubeflow Pipelines, Seldon Core or BentoML (model serving), Kubernetes, Prometheus (model metrics), Grafana (drift dashboards), DVC (data versioning), GitHub Actions

Implementation Steps

1. Instrument training code with MLflow: log parameters, metrics, and artifacts for every experiment run.
2. Set up MLflow Model Registry backed by S3 for artifact storage and PostgreSQL for metadata.
3. Define a model promotion workflow: Staging → Production requires automated performance tests and human approval.
4. Package models as Docker containers using BentoML; include preprocessing, inference, and postprocessing in a single service.
5. Deploy models to Kubernetes using a Deployment and HPA; expose via a Kubernetes Service and Ingress.
6. Instrument the model serving container with Prometheus metrics: prediction latency, input distribution statistics.
7. Implement data drift detection using Evidently AI; alert when feature distributions shift beyond thresholds.
8. Build a model performance dashboard in Grafana showing prediction accuracy, latency, and drift metrics over time.

Best Practices

- Version both the model and the training data; reproducing a model without versioned data is impossible.
- Define model performance SLOs: prediction accuracy and latency thresholds below which a model is considered degraded.
- Shadow test new model versions: run new and old models in parallel; compare predictions before routing live traffic.
- Implement canary deployments for models: send 5% of traffic to the new model version and compare accuracy on live data.
- Automate model retraining triggers: schedule weekly retraining or trigger on data drift detection.

Challenges and Solutions

Challenge Model inference latency is too high for real-time API SLOs.	Solution Profile inference bottlenecks; use ONNX Runtime for optimised inference, GPU nodes for deep learning models, and response caching for common inputs.
Challenge Training data is not reproducible between runs, causing inconsistent model performance.	Solution Version all datasets with DVC pointing to S3; pin dataset versions in model training configuration.
Challenge MLflow model registry grows unwieldy with hundreds of experiment runs.	Solution Implement MLflow run cleanup automation: retain the top 10 runs per experiment and archive the rest to cold storage.

Project 45: Network Policy Automation with Cilium eBPF

Project 45 of 50 • Networking & Service Mesh

Description

Deploy Cilium as the Kubernetes CNI (Container Network Interface) using eBPF for high-performance, policy-aware networking. Replace iptables-based network policies with Cilium's expressive policies that can filter on DNS names, HTTP paths, and service identities rather than just IP addresses and ports.

Use Case

Platform teams that have outgrown Kubernetes NetworkPolicy limitations and need application-layer network security. Cilium enables micro-segmentation based on workload identity and HTTP method, supporting zero-trust network models at the layer-7 level.

Tools and Technologies

Cilium, Hubble (Cilium observability), Kubernetes, Prometheus (Cilium metrics), Grafana, cilium-cli, Hubble UI, Tetragon (eBPF-based runtime security)

Implementation Steps

1. Replace the existing CNI with Cilium during a maintenance window; use cilium install via Helm.
2. Enable Hubble for network observability: deploy Hubble Relay and Hubble UI.
3. Write CiliumNetworkPolicy resources to replace existing Kubernetes NetworkPolicy with equivalent rules.
4. Implement DNS-based egress policies: allow pods to reach api.external-service.com by DNS name, not by IP.
5. Write L7 HTTP policies: allow GET /api/v1/products but deny DELETE /api/v1/products from the frontend service.
6. Use Hubble UI to visualise service communication flows and identify unexpected traffic.
7. Configure Prometheus scraping for Cilium and Hubble metrics; build Grafana dashboards for network throughput and policy drops.
8. Deploy Tetragon for kernel-level runtime security enforcement alongside Falco.

Best Practices

- Start by observing network flows in Hubble before writing policies; base policies on actual traffic, not assumptions.
- Implement a default-deny policy at the namespace level; add allow rules explicitly for each required flow.
- Use Cilium's ClusterMesh for policy-consistent cross-cluster communication in multi-cluster environments.
- Monitor policy drop metrics in Grafana; unexplained drops indicate misconfigured policies or unexpected traffic.
- Version control all CiliumNetworkPolicy resources in the GitOps repository alongside application manifests.

Challenges and Solutions

Challenge Cilium node agent crashes cause network interruptions on affected nodes.	Solution Deploy with high availability settings; monitor Cilium agent health via DaemonSet metrics and alert on agent restarts.
Challenge DNS-based policies have latency for initial DNS lookups.	Solution Pre-warm DNS entries using Cilium FQDN pre-fetch; accept that the first connection to a new DNS target incurs a brief delay.
Challenge Migrating from a different CNI to Cilium requires careful planning.	Solution Use Cilium's migration guide; perform the migration in stages starting with non-production clusters; schedule a maintenance window for production.

Project 46: Automated Certificate Management with cert-manager

Project 46 of 50 • Security & DevSecOps

Description

Deploy cert-manager on Kubernetes to fully automate TLS certificate issuance, renewal, and rotation for all internal and external services. Certificates are issued from Let's Encrypt or a private CA, renewed automatically 30 days before expiry, and mounted into pods as Kubernetes Secrets without any manual intervention.

Use Case

Teams that have experienced production outages from expired TLS certificates. Manual certificate management at scale is error-prone and time-consuming. cert-manager eliminates certificate expiry incidents entirely by automating the full lifecycle.

Tools and Technologies

cert-manager, Let's Encrypt (ACME), AWS ACM (Private CA), HashiCorp Vault PKI (internal CA), Kubernetes, Ingress-nginx or ALB Ingress Controller, Prometheus (certificate expiry metrics), Grafana

Implementation Steps

1. Install cert-manager using Helm; configure the cert-manager namespace and required CRDs.
2. Create a ClusterIssuer for Let's Encrypt using DNS-01 challenge (AWS Route 53 integration for wildcard certs).
3. Create a ClusterIssuer for the internal CA backed by Vault PKI for internal service certificates.
4. Annotate Ingress resources with cert-manager annotations; cert-manager issues and renews certificates automatically.
5. For internal services, create Certificate resources specifying the Issuer, DNS names, and secret name.
6. Configure Prometheus to scrape cert-manager metrics; alert if any certificate is less than 14 days from expiry.
7. Test renewal by deleting the TLS secret for a non-critical service; cert-manager should reissue within minutes.
8. Implement certificate rotation testing in CI: verify that applications reload certificates without restarting.

Best Practices

- Prefer DNS-01 challenge over HTTP-01 for wildcard certificates and services not exposed on port 80.
- Use Let's Encrypt staging issuer for testing to avoid rate limit exhaustion on the production ACME server.
- Alert at 14 days remaining, not 0; cert-manager should renew at 30 days, but having a human alert at 14 days provides a safety net.
- Use a private CA for internal service certificates; do not expose internal services with public CA certificates.
- Document the certificate inventory: which services use which issuer, and why — especially for any manually managed certs.

Challenges and Solutions

Challenge

DNS-01 challenge fails because Route 53 API calls are rate-limited.

Solution

Spread certificate issuance; avoid bulk certificate creation that triggers multiple Route 53 API calls simultaneously. Use propagation wait settings.

Challenge

Applications cache TLS certificates in memory and do not reload when secrets update.

Solution

Use cert-manager's certificate rotation trigger to restart pods when certificates are renewed, or implement inotify-based secret watching in the application.

Challenge

cert-manager webhook is unavailable during cluster upgrades, blocking admission.

Solution

Ensure cert-manager replicas are spread across nodes; configure PodDisruptionBudget to prevent all replicas draining simultaneously.

Project 47: Automated Compliance Reporting for SOC 2 / ISO 27001

Project 47 of 50 • Security & DevSecOps

Description

Build an automated compliance evidence collection and reporting pipeline for SOC 2 Type II and ISO 27001 audits. Every control is mapped to an automated check that runs continuously; evidence is collected daily and formatted into audit-ready reports, reducing audit preparation time from months to days.

Use Case

Companies pursuing their first SOC 2 Type II certification or maintaining an existing one. Manual evidence collection for dozens of controls across hundreds of systems is a multi-month effort. Automation makes compliance continuous and evidence always current.

Tools and Technologies

Drata or Vanta (compliance automation), AWS Config Rules, CloudTrail, GitHub (access logs), Jira (evidence tracker), Python (report generation), Slack (compliance alerts), Terraform (control deployment)

Implementation Steps

1. Map all SOC 2 Trust Services Criteria to technical controls; identify which can be automated versus manually evidenced.
2. Enable AWS CloudTrail with log file integrity validation; ship logs to a compliance S3 bucket with object lock.
3. Write AWS Config rules for each automated control: MFA enabled, EBS encryption at rest, S3 public access blocked.
4. Deploy Drata or Vanta to aggregate evidence from AWS, GitHub, Jira, and HR systems automatically.
5. Create a Jira board for compliance exceptions: every failing control creates a ticket with the remediation owner.
6. Schedule a weekly compliance summary report: email to CISO and engineering leads showing control status by category.
7. Build a security awareness training tracker: integrate with the HR system to verify all staff complete training.
8. Conduct quarterly access reviews: automated script compares IAM users and GitHub access against the HR system and flags discrepancies.

Best Practices

- Aim for 80%+ automated controls; fully manual controls are the first to lapse during busy periods.
- Store all evidence in an immutable, auditor-accessible location; auditors must trust that evidence cannot be retroactively modified.
- Assign control owners: every control has an owner who is accountable for remediation when it fails.
- Run a mock audit quarterly to identify gaps before the real auditor arrives.
- Use compliance findings to drive security improvement, not just audit paperwork; every finding is a real risk.

Challenges and Solutions

Challenge

Automated controls cover infrastructure but evidence for people controls (training, access reviews) is still manual.

Solution

Integrate with Workday or BambooHR for training records; use GitHub API for access review data; build composite evidence bundles automatically.

Challenge

Compliance evidence collection tool has limited integrations with internal tools.

Solution

Build custom evidence collectors as Python scripts that run on schedule and upload structured evidence packages to the compliance platform via API.

Challenge

SOC 2 scope creep: auditors request evidence for systems not in the original scope.

Solution

Define and document the scope boundary at the start of the audit cycle; have legal review and agree the scope with the auditor before evidence collection begins.

Project 48: Progressive Web App Deployment Pipeline

Project 48 of 50 • Continuous Integration & Delivery

Description

Build a comprehensive CI/CD pipeline for a Progressive Web App (PWA) including bundle size budgets, Lighthouse performance audits, cross-browser testing, CDN deployment, and service worker cache invalidation. Every release is automatically validated against performance and accessibility standards before reaching users.

Use Case

Frontend engineering teams that need to maintain performance standards as a PWA codebase grows. Bundle size bloat, Lighthouse score regressions, and broken service worker updates are common issues that automated pipelines can catch before they reach users.

Tools and Technologies

GitHub Actions, Webpack / Vite, Jest / Cypress, Playwright (cross-browser), Lighthouse CI, Bundlesize, Cloudflare Pages / AWS S3 + CloudFront, Service Worker tooling (Workbox)

Implementation Steps

1. Configure bundle size budgets in `bundlesize.config.js`; fail the CI pipeline if any bundle exceeds its budget.
2. Set up Lighthouse CI with a configuration file defining minimum scores: Performance 85, Accessibility 90, Best Practices 90, SEO 80.
3. Run Playwright tests against Chrome, Firefox, and Safari (WebKit) in the CI pipeline using matrix strategy.
4. Deploy to a staging environment (Cloudflare Pages preview or S3 staging bucket) automatically on every PR.
5. Run Lighthouse CI against the staging deployment; post the score as a PR comment.
6. On merge to main, build production assets with content hashing for cache busting.
7. Deploy to CloudFront; create a cache invalidation for `index.html` and `service-worker.js`.
8. Run a smoke test against production after deployment: verify critical user journeys with Playwright.

Best Practices

- Cache bust aggressively: all assets except `index.html` and `service-worker.js` should be immutably cached (Cache-Control: `max-age=31536000, immutable`).
- Test service worker updates explicitly: ensure the update flow shows users a prompt and reloads without data loss.
- Monitor Core Web Vitals in production using the `web-vitals` library; send metrics to your analytics platform.
- Set bundle size budgets per chunk, not just total size; lazy-loaded routes that exceed a threshold indicate code bloat.
- Enable Brotli compression on CloudFront; it reduces bundle sizes by 15–20% compared to gzip.

Challenges and Solutions

Challenge

Lighthouse scores vary between CI and production due to server warm-up and CDN caching.

Solution

Run Lighthouse CI 3 times and take the median; exclude first-run cold start from threshold evaluation.

Challenge

Service worker cached content causes users to see stale versions after deployment.

Solution

Use Workbox's background sync and update-on-reload strategy; test the update flow with Playwright by simulating an app update.

Challenge

Cross-browser tests in CI are slow (30+ minutes) blocking rapid iteration.

Solution

Run full cross-browser suite only on main branch merges; run Chrome-only tests on PR branches for fast feedback.

Project 49: Automated Kubernetes Security Scanning with Kube-bench and Kube-hunter

Project 49 of 50 • Security & DevSecOps

Description

Implement continuous Kubernetes security assessment using kube-bench (CIS Benchmark checks) and kube-hunter (active vulnerability scanning). Run assessments on a schedule and on every cluster configuration change, tracking remediation progress in a dashboard over time.

Use Case

Platform and security teams responsible for the security of shared Kubernetes infrastructure. CIS Kubernetes Benchmark compliance is required for many certifications, and kube-hunter actively identifies exploitable paths that configuration reviews miss.

Tools and Technologies

kube-bench, kube-hunter, Kubernetes, GitHub Actions, Elasticsearch (findings storage), Grafana (benchmark dashboard), Slack notifications, Jira (remediation tracking)

Implementation Steps

1. Run kube-bench as a Job inside the cluster; it checks control plane and node configurations against the CIS Benchmark.
2. Schedule kube-bench weekly and after every cluster upgrade using a CronJob.
3. Parse kube-bench JSON output with a Python script; categorise findings by severity and component.
4. Store findings in Elasticsearch; build a Grafana dashboard tracking pass/fail counts over time.
5. Run kube-hunter in network-mode from outside the cluster to simulate an external attacker perspective.
6. Run kube-hunter in pod-mode from within the cluster to simulate a compromised pod perspective.
7. Create a Jira epic for CIS Benchmark remediation; one ticket per failing check with the remediation steps.
8. Integrate findings into the security posture dashboard alongside Falco, CSPM, and image scanning results.

Best Practices

- Remediate CIS Benchmark findings by criticality: fix Level 1 (automated, low-risk) findings first before tackling Level 2.
- Document accepted exceptions: some CIS checks may not apply to your environment; record the justification.
- Run kube-hunter from a CI/CD runner outside the cluster to verify that the cluster is not reachable from unexpected network paths.
- Set a target CIS compliance score (e.g., 80% pass rate on Level 1); track progress monthly.
- Share benchmark results with the development teams; platform security is a shared responsibility.

Challenges and Solutions

Challenge

Some kube-bench checks require access to control plane nodes, which is unavailable on managed EKS.

Solution

Run only the applicable kube-bench profile (eks) which excludes checks requiring direct master node access; supplement with AWS Security Hub EKS best practices checks.

Challenge

kube-hunter active scanning generates noise in network security tools.

Solution

Schedule kube-hunter with security team awareness; run from a known scanner IP that is excluded from IDS alert rules.

Challenge

Remediating control plane settings requires EKS API calls that may trigger cluster updates.

Solution

Test all kube-bench remediations in a staging cluster first; some settings require node replacement rather than in-place changes.

Project 50: Full-Stack Observability with OpenTelemetry and Grafana Stack

Project 50 of 50 • Observability

Description

Implement a unified observability platform combining metrics, logs, and traces using the OpenTelemetry standard and the Grafana LGTM stack (Loki for logs, Grafana for dashboards, Tempo for traces, Mimir for metrics). Engineers have correlated visibility across all three pillars from a single pane of glass.

Use Case

Engineering teams that use separate tools for metrics, logs, and traces and waste time correlating data manually during incidents. A unified observability stack with correlated signals reduces mean time to resolution, as engineers can jump from a metric spike to the relevant trace to the underlying log entry in seconds.

Tools and Technologies

OpenTelemetry Collector, Grafana (dashboards), Loki (logs), Tempo (traces), Mimir (long-term metrics), Prometheus, Alloy (OpenTelemetry agent), Kubernetes, Helm

Implementation Steps

1. Deploy the Grafana LGTM stack using the community Helm chart (grafana/lgtm-distributed) or individual charts.
2. Deploy Grafana Alloy as the OpenTelemetry collector DaemonSet; configure pipelines for logs, metrics, and traces.
3. Configure Alloy to scrape Prometheus metrics, collect container logs via the Kubernetes log API, and receive OTLP traces.
4. Route all data through Alloy: metrics to Mimir, logs to Loki, traces to Tempo.
5. Instrument all services with the OpenTelemetry SDK generating correlated trace IDs in all three signals.
6. Build Grafana dashboards using the Explore view to demonstrate log-trace-metric correlation: click a trace ID in logs to jump to Tempo.
7. Create RED method dashboards per service: Request rate, Error rate, Duration (using Mimir data).
8. Set up Grafana OnCall (or PagerDuty integration) for alert routing from unified Grafana alerting rules.

Best Practices

- Inject the same trace ID into structured logs, metric exemplars, and spans to enable 3-way correlation.
- Use OpenTelemetry semantic conventions for attribute names; consistency across services enables cross-service queries.
- Configure retention tiers: high-resolution metrics for 15 days, aggregated for 1 year; full traces for 7 days, sampled for 30 days.
- Use Mimir's ruler for alerting rules so all alert definitions are in a single place alongside dashboards.
- Test observability during GameDay exercises: engineers should be able to identify root cause from dashboards alone, without SSH access.

Challenges and Solutions

Challenge OpenTelemetry Collector pipeline configuration is complex and errors cause silent data loss.	Solution Enable the debug exporter during initial setup to verify data flow; monitor Collector self-metrics (dropped spans, failed exports) in Grafana.
Challenge Loki query performance degrades with large log volumes and high cardinality labels.	Solution Use structured metadata for high-cardinality fields; keep Loki labels low-cardinality (namespace, pod, level only). Use LogQL pipeline filters for the rest.
Challenge Mimir ingestion costs and storage grow faster than expected.	Solution Profile metric cardinality using mimirtool; drop high-cardinality metrics at the Alloy level before they reach Mimir.